

# A Comprehensive View on TD Prevention Practices and Reasons for Not Preventing It

SÁVIO FREIRE, Federal Institute of Ceará, Morada Nova and Federal University of Bahia, Salvador, Brazil
ALEXIA PACHECO, University of Costa Rica, San José, Costa Rica
NICOLLI RIOS, Federal University of Rio de Janeiro, Rio de Janeiro, Brazil
BORIS PÉREZ, Francisco de Paula Santander University, Cúcuta, Colombia
CAMILO CASTELLANOS and DARÍO CORREAL, University of Los Andes, Bogotá, Colombia
ROBERT RAMAČ, VLADIMIR MANDIĆ, and NEBOJŠA TAUŠAN, University of Novi Sad, Novi Sad, Serbia
GUSTAVO LÓPEZ, University of Costa Rica, San José, Costa Rica
MANOEL MENDONÇA, Federal University of Bahia, Salvador, Brazil
DAVIDE FALESSI, University of Rome Tor Vergata, Rome, Italy
CLEMENTE IZURIETA, Montana State University, Bozeman, MT, USA
CAROLYN SEAMAN, University of Maryland Baltimore County, Baltimore, MD, USA
RODRIGO SPÍNOLA, Virginia Commonwealth University, Richmond, VA, USA

*Context.* Technical debt (TD) prevention allows software practitioners to apply practices to avoid potential TD items in their projects. *Aims.* To uncover and prioritize, from the point of view of software practitioners, the practices that could be used to avoid TD items, the relations between these practices and the causes of TD, and the practice avoidance reasons (PARs) that could explain the failure to prevent TD. *Method.* We analyze data collected from six replications of a global industrial family of surveys on TD, totaling 653 answers. We also conducted a follow up survey to understand the importance level of analyzed data. *Results.* Most practitioners indicated that TD could be prevented, revealing 89 prevention practices and 23 PARs for explaining the failure to prevent TD. The article identifies statistically significant relationships between preventive practices and

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) Finance Code 001 and the Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq).
Author's Contact Information: Sávio Freire, Federal Institute of Ceará, Morada Nova and Federal University of Bahia, Salvador, Brazil; e-mail: savio.freire@ifce.edu.br; Alexia Pacheco, University of Costa Rica, San José, Costa Rica; e-mail: alexia.pacheco@ucr.ac.cr; Nicolli Rios, Federal University of Rio de Janeiro, Rio de Janeiro, Brazil; e-mail: nicolli@cos.ufrj.br; Boris Pérez, Francisco de Paula Santander University, Cúcuta, Colombia; e-mail: br.perez41@uniandes.edu.co; Camilo Castellanos, University of Los Andes, Bogotá, Colombia; e-mail: cc.castellanos87@uniandes.edu.co; Darío Correal, University of Los Andes, Bogotá, Colombia; e-mail: dcorreal@uniandes.edu.co; Robert Ramač, University of Novi Sad, Novi Sad, Serbia; e-mail: ramac.robert@uns.ac.rs; Vladimir Mandić, University of Novi Sad, Novi Sad, Serbia; e-mail: vladman@uns.ac.rs; Nebojša Taušan, University of Novi Sad, Novi Sad, Serbia; e-mail: nebojsa.tausan@ef.uns.ac.rs; Gustavo López, University of Costa Rica, San José, Costa Rica; e-mail: gustavo.lopez_h@ucr.ac.cr; Manoel Mendonça, Federal University of Bahia, Salvador, Brazil; e-mail: manoel.mendonca@ufba.br; Davide Falessi, University of Rome Tor Vergata, Rome, Italy; e-mail: falessi@ing.uniroma2.it; Clemente Izurieta, Montana State University, Bozeman, MT, USA; e-mail: clemente.izurieta@montana.edu; Carolyn Seaman, University of Maryland Baltimore County, Baltimore, MD, USA; e-mail: cseaman@umbc.edu; Rodrigo Spínola (Corresponding author), Virginia Commonwealth University, Richmond, VA, USA; e-mail: spinolaro@vcu.edu
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7392/2024/9-ART178
https://doi.org/10.1145/3674727

certain causes of TD. Further, it prioritizes the list of practices, PARs, and relationships regarding their level of importance for TD prevention based on the opinion of software practitioners. *Conclusion.* This work organizes TD prevention practices and PARs in a conceptual map and the relationships between practices and causes of TD in a Sankey diagram to help the visualization of the body of knowledge reported in this study.

CCS Concepts: • **General and reference** → **Empirical studies**; • **Software and its engineering** → **Maintaining software**;

Additional Key Words and Phrases: Technical debt, technical debt prevention, technical debt management, InsighTD project

**ACM Reference format:**
Sávio Freire, Alexia Pacheco, Nicolli Rios, Boris Pérez, Camilo Castellanos, Darío Correal, Robert Ramač, Vladimir Mandić, Nebojša Taušan, Gustavo López, Manoel Mendonça, Davide Falessi, Clemente Izurieta, Carolyn Seaman, and Rodrigo Spínola. 2024. A Comprehensive View on TD Prevention Practices and Reasons for Not Preventing It. *ACM Trans. Softw. Eng. Methodol.* 33, 7, Article 178 (September 2024), 44 pages.
https://doi.org/10.1145/3674727

## 1 Introduction

**Technical debt (TD)** refers to postponed tasks, immature artifacts, or even mistakes in software projects [22, 41]. Incurring debt can bring short-term benefits, like high productivity, but also long-term drawbacks, making the software difficult to evolve [11, 19, 49, 68]. To make TD items visible and under control, TD management intends to balance benefits and drawbacks of debt presence [65]. TD prevention is one of the key TD management activities [33, 65]. It aims to avoid the occurrence of potential TD items [33, 65]. For example, software teams can improve their current development processes to curb the occurrence of specific types of debt [39].

Several studies have investigated strategies and practices for TD prevention. Codabux et al. [64] proposed a set of prevention practices for agile software development. Li et al. [65] defined four categories of TD prevention from a systematic mapping study. Yli-Huumo et al. [23], Yli-Huumo et al. [24], and Gupta et al. [43] performed case studies, while Silva et al. [59], Apa et al. [12, 13] conducted surveys to investigate practices used in TD prevention. Rios et al. [37] and Aragão et al. [9] investigated the prevention practices for curbing documentation and test debt items, respectively. All these studies are grounded on small sample sizes or few organizations and are not explicitly focused on TD prevention.

Although these studies provide evidence on TD prevention, the subject deserves a more comprehensive investigation. Overall, it is fair to expect that TD prevention can sometimes be cheaper than its repayment. Moreover, prevention may also help other TD management activities. For example, setting up prevention practices helps in catching inexperienced developers' "not-so-good" solutions [24]. Further, having information on TD prevention practices can guide software practitioners in choosing practices that could be used in their projects, learning from the experience of practitioners who have applied these practices.

However, just having information about TD prevention practices is not enough to succeed in TD prevention initiatives. In general, software teams need the support of their managers or organizations to spend effort on complementary activities in their projects. Without managerial or organizational support, a TD prevention initiative may fail to enact prevention practices even when the development team is aware of them. It is crucial to recognize the factors that can hinder the execution of the TD prevention activities and to define strategies to minimize their effects.

This work aims to uncover and prioritize, from the point of view of software practitioners, the practices that could be used to avoid TD items, the relations between these practices and the causes of TD, and the **practice avoidance reasons (PARs)** that could explain the failure to prevent TD. Developers, testers, requirements engineers, and managers focus on diverse TD items and can indicate different practices to avoid TD items. Therefore, we consider practitioners who perform different roles in software development processes.

To this end, we use a large data set extracted from the InsighTD project, a globally distributed family of surveys that investigates the state of the practice on TD, its causes, effects, and management practices [36]. The data set we use is composed of 653 answers given by practitioners from the Brazilian, Chilean, Colombian, Costa Rican, North American, and Serbian software industries (hence the large number of co-authors in the article). This work substantially expands prior InsighTD analyses on TD prevention by providing:

- An up-to-date comprehensive list of practices related to TD prevention ranked by the frequency of their citation by software practitioners.
- An up-to-date comprehensive list of PARs ranked by the frequency of their citation by software practitioners.
- A set of relationships between TD prevention-related practices and causes of TD, revealing what practices could be more appropriate to prevent a specific cause.
- A Sankey diagram [44] that organizes the relationships between TD prevention-related practices and causes of TD, considering the relative risk [14] as a criterion for matching prevention practices with specific causes of debt.
- A categorization of TD practices and PARs with respect to their relationship to software planning, development, management, and methodology issues.
- A conceptual map that organizes TD practices and PARs in their respective categories and types.

The findings provided here can support software practitioners in handling TD items by considering the experience of other software practitioners in preventing TD. The ranked lists of practices and PARs can support the decision of what practice to implement or what PARs curb the implementation of TD prevention. The conceptual map serves as a reference that software practitioners can consult when they are interested in applying practices to curb the existence of TD items or understanding the reasons that lead to the non-implementation of prevention practices. It can also help researchers in identifying new themes of research over the state of the practice it maps out. The Sankey diagram can support software practitioners in revealing practices already used to prevent specific causes of TD.

This article is organized as follows. Section 2 presents related work on TD prevention and causes of TD. In Section 3, we present the research method. Section 4 presents the obtained results. Section 5 discusses these results, presenting the conceptual map and the Sankey diagram. Section 6 discusses the study's validity. Lastly, Section 7 presents our final remarks.

## 2 Related Work

This section presents the related work on TD prevention and causes of TD. Also, it discusses our prior studies on these topics.

### 2.1 TD Prevention

Several authors have investigated the practices to prevent debt items. Li et al. [65] defined the following categories of TD prevention: *development process improvement*, *architecture decision making support*, *lifecycle cost planning*, and *human factors analysis*. By conducting a case study in

software industries, Yli-Huumo et al. [23] and Yli-Huumo et al. [24] revealed some practices to prevent debt items, such as *code reviews*, *code standards*, and *bug fixing days*. Codabux et al. [64] identified that prevention practices can prevent the debt (e.g., *education and training*) or contribute to TD prevention (e.g., *bugs bash*). Only *bugs bash* practice was confirmed by Gupta et al. [43]. Silva et al. [59], Apa et al. [12, 13] also reported prevention practices, such as *guidelines*, *coding standards*, *code revisions*, *retrospective meetings*, and *definition of done*.

Lastly, approaching specific types of debt, Rios et al. [37] organized the set of preventive practices for documentation debt in a conceptual map. Practices like *comment the code* and *have a documentation repository* were reported. In their turn, Aragão et al. [9] defined a catalog to support the management of test debt items, including their prevention. Examples of prevention practices for test debt are *present already identified debts* and *review elaborated test cases*.

In summary, the related studies contribute to painting a view of TD prevention in the software industry. We will compare them with our results based on InsighTD survey data and observe how they complement each other. The result of the comparison is described in Section 4.2.

*2.1.1 Prior Studies on TD Prevention from the InsighTD Project.* We have published several results from InsighTD project. Table 1 summarizes our previous publications with respect to the publication venue (conference or journal), research focus, and viewpoint. The research focus is related to the investigation of the TD concept, the causes and effects of TD, and TD management (payment, prevention, and monitoring). The viewpoint used in the analysis (last column) can be (1) *comprehensive*, i.e., addressing something broadly associated with the TD concept, e.g., defining tools and strategies for TD management [17]; or (2) *specific*, such as studying a specific TD type, e.g., *requirements debt* [27], a software development activity, e.g., *software testing* [28], a process model, e.g., *Agile* [34] or a role, e.g., *software architects* [7].

Seven studies from the InsighTD project (rows 6, 14, 16, 19, 20, 22, and 25 in Table 1) have investigated TD prevention in some way. Most of these studies used a subset of the InsighTD data, where the data was chosen to represent a particular type of debt (e.g., Barbosa et al. [27] focused on requirements and requirements documentation debt), or on responses from Agile environments (e.g., Freire et al. [53]), or on responses from a particular type of respondent (e.g., Pérez et al. [7] focused on software architects). Other studies [38, 50] were limited to the InsighTD data available at the time. Berenguer et al. [17, 18] used the full InsighTD data set, as we do, but did not focus on TD prevention. Although, they discussed whether TD prevention is more related to coding or other software development issues.

The analysis described in this article builds on the work described by Freire et al. [50, 53], both of which investigated TD prevention in a comprehensive way on a restricted InsighTD data set. These two studies identified and categorized prevention practices and PARs. The present work goes further by increasing the confidence of these results with more data from different countries and from non-Agile environments. Moreover, we analyze the relationships between causes of TD and the identified preventive practices, also investigating the importance level of these relationships, practices, and PARs by conducting a reduced-scope (i.e., more focused) survey with a subset of the survey respondents.

In summary, most of our prior studies on TD prevention presented results limited to a specific role or process model. By using the entire InsighTD data set, the present work increases the validity of our prior studies, i.e., the extent to which these findings approximate other real-world scenarios [15]. It also performs a much broader set of analyses, including:

(1) A detailed analysis of the TD prevention-related practices and PARs involving:
   (a) The updated list of practices/PARs (see Sections 4.2 and 4.6).

Table 1. Summary of the InsighTD Publications

| No. | References | Conference (C)/journal (J) | TD research focus | | | | | Viewpoint considered during analyses |
|---|---|---|---|---|---|---|---|---|
| | | | Concept | Causes/effects | Payment | Prevention/prevention avoidance | Monitoring | |
| 1 | Rios et al. [32] | C | | ○ | | | | Comprehensive |
| 2 | Pacheco et al. [4] | C | ○ | | | | | Comprehensive |
| 3 | Pérez et al. [5] | C | | ○ | | | | Comprehensive |
| 4 | Rios et al. [34] | C | | ○ | | | | Agile software projects |
| 5 | Rios et al. [35] | C | | ○ | | | | Comprehensive |
| 6 | Freire et al. [50] | C | | | | ◯ | | Comprehensive |
| 7 | Freire et al. [51] | C | | | ◯ | | | Comprehensive |
| 8 | Mandić et al. [60] | C | ○ | | | | | Comprehensive |
| 9 | Pérez et al. [6] | C | | | ◯ | | | Comprehensive |
| 10 | Ramač et al. [46] | C | | | ○ | | | Comprehensive |
| 11 | Rios et al. [36] | J | ○ | ○ | | | | Comprehensive |
| 12 | Rios et al. [37] | C | | ◯ | | | | Documentation debt |
| 13 | Souza et al. [28] | C | | ○ | | | | Software testing |
| 14 | Berenguer et al. [17] | C | ● | | | ● | | Comprehensive |
| 15 | Freire et al. [52] | C | | ● | | | | Experience level |
| 16 | Freire et al. [53] | J | | | ● | ● | ● | Agile software projects |
| 17 | Freire et al. [54] | C | | ◯ | ◯ | | | Comprehensive |
| 18 | Mandić et al. [61] | J | | ● | ● | | | Experience level |
| 19 | Pérez et al. [7] | J | | | ◯ | ◯ | | Software architects |
| 20 | Rios et al. [38] | J | | | ◯ | ◯ | ◯ | Process models |
| 21 | Rocha et al. [58] | C | | ◯ | | | | Software testing |
| 22 | Barbosa et al. [27] | C | | ◯ | | ◯ | | Requirements debt |
| 23 | Ramač et al. [47] | J | ● | ● | | | | Comprehensive |
| 24 | Ramač et al. [48] | C | | | ○ | | | Comprehensive |
| 25 | Berenguer et al. [18] | J | ● | | | ● | | Comprehensive |
| 26 | Freire et al. [55] | J | | | ● | | | Comprehensive |

Symbols indicate the data set used: (○) data from a single replication, (◯) data from several replications, and (●) data from all replications.

(b) The list of practices or PARs ranked by the level of importance in software practitioners' opinion (see Sections 4.3 and 4.7).
(c) The relationships between practices and causes of debt (see Section 4.4).

(d) The list of relationships between practices and causes of debt ranked by the level of importance in software practitioners' opinion (see Section 4.5).
(2) A conceptual map encompassing the practices and PARs (see Section 5.2).
(3) A Sankey diagram including the relationships between practices and causes of debt (see Section 5.3).
(4) A comparison of the results with related work (see Section 4.2).

The added analyses and the produced artifacts (the map and Sankey diagram) can support software practitioners and researchers in performing their activities by understanding the practices used in the software industry to curb the presence of TD items and the PARs that hamper the application of these practices. The artifacts open new avenues of research and help practitioners in defining their own strategy for TD prevention, e.g., by selecting a practice or a set of practices tailored to their project context.

### 2.2 Causes of TD

Codabux and Williams [63] found that *resource constraints* (such as a *limited timeframe* and the *unavailability of developers to continue working on the feature*) are causes of TD in the perception of agile software practitioners. Yli-Huumo et al. [23] and Martini et al. [3] identified the main causes of TD in general (e.g., *lack of time given for development* and *lack of coding standards and guides*) and of architectural debt (*reuse of legacy/third party/open source* and *non-completed refactoring*), respectively. Li et al. [65] identified in their systematic review that causes of TD are associated with quality aspects. Ernst et al. [30] revealed that *immature choices done in architecture* are a key cause of TD. Avgeriou et al. [40] and Izurieta et al. [20] consolidated the perception of several researchers and practitioners on the causes of TD, indicating that they can be a *process*, a *decision*, an *action(or lack thereof)*, or an *event that triggers the existence of that TD item*, such as *schedule pressure*, *unavailability of a key person*, or *lack of information about a technical feature*. Lastly, Rios et al. [31] reported that TD causes occur in both isolation and combined ways.

As seen in Table 1, the InsighTD Project has also produced a sizeable body of knowledge on TD causes and effects. Among other results, Freire et al. [52] reported that the perception of causes and effects is influenced by the practitioners' experience, and Ramač et al. [47] discussed the causes of TD by considering the full InsighTD data set. We reviewed their findings, resulting in a slight update on the rank of causes. Table 2 summarizes the 10 most commonly cited causes, reporting the cause name and the total number (i.e., count) of citations (#CC). The column %PC presents the percentage of #CC in relation to the total of projects, revealing how frequently each cause was identified in software projects. The complete list of causes is available in the complementary material [56].

The related work and our previous work on TD causes still miss a discussion on the relationship between prevention practices and TD causes. This article fills this gap and sheds light on how better to choose the use of specific preventive practices in the presence of specific TD causes.

## 3 Research Method

This section presents the **research questions (RQs)** the data collection and analysis approach we followed to answer the questions.

### 3.1 RQs

Our research goal is *to uncover and prioritize, from the point of view of software practitioners, the practices that could be used to avoid TD items, the relations between these practices and the causes of*

Table 2. Top 10 TD Causes

| No | cause | #CC[a] | %PC[b] |
|---|---|---|---|
| 1st | Deadline | 169 | 26% |
| 2nd | Not effective project management | 98 | 15% |
| 3rd | Inappropriate planning | 83 | 13% |
| 4th | Lack of technical knowledge | 80 | 12% |
| 5th | Focus on producing more at the expense of quality | 67 | 10% |
| 6th | Inappropriate/poorly planned/poorly executed test | 59 | 9% |
| 7th | Lack of experience | 58 | 9% |
| 8th | Inaccurate time estimate | 56 | 9% |
| 9th | Lack of qualified professional | 54 | 8% |
| 10th | Non-adoption of good practices | 54 | 8% |

Adapted from Ramač et al. [47].
[a]Count of causes.
[b]Percentage of CC in relation to the total of all projects (642).

Table 3. RQs

| ID | RQ |
|---|---|
| RQ1 | What are the leading prevention practices that can be used to avoid TD? |
| RQ1.1 | How do software practitioners perceive the importance of the prevention practices? |
| RQ2 | How do prevention practices relate to causes of TD in software projects? |
| RQ2.1 | How do software practitioners perceive the importance of the identified relations? |
| RQ3 | What are the PARs considered by software practitioners to explain the failure to prevent TD? |
| RQ3.1 | How do software practitioners perceive the importance of these PARs? |

*TD, and the reasons that could explain the failure to prevent TD*. To achieve this goal, we define a set of RQs, as shown in Table 3.

The purpose of *RQ1* is to identify the preventive practices that software practitioners can apply to curb the presence of TD items. Also, through this question, we update the list of preventive practices reported in Freire et al. [50], considering data from six InsighTD replications totaling 546 answers (the previous work considered two replications with 182 answers). In *RQ1.1*, we go further to investigate which practices are more important to prevent TD items, i.e., which practices are more decisive for supporting TD prevention. Knowing the most important practices can support software teams in defining prevention strategies, considering the experience of other practitioners, which can be especially important in the early stages of the definition of such strategies.

*RQ2* aims to identify the practices that can be used, in the presence of specific causes, to prevent TD. We also investigate the importance of each relationship in *RQ2.1*, i.e., how decisive each prevention practice is to prevent a debt item incurred due to a cause. Having information on the most important relationships between prevention practices and causes of TD can support software practitioners in implementing prevention practices that focus on particular TD causes existing in their projects. Lastly, *RQ3* seeks to investigate the PARs that can be used to explain the failure to prevent TD. Further, we intend to identify which PARs are more important in the practitioners' opinion in *RQ3.1*. By answering these questions, software teams can improve their perception on PARs and change their mindset on TD prevention, making their environment more favorable to avoid TD items.

### 3.2 A Two-Phase Survey Study

We use two surveys to answer the RQs: the InsighTD survey and a follow-up survey specifically designed for the scope of our study. We decided to use surveys, as they take a snapshot of a determinate situation using sample questioning to understand a population [16]. A survey is a convenient research instrument for collecting responses from large samples and can help to provide broad view of large populations. One must keep in mind, however, that surveys have some drawbacks. In a survey, data about a phenomenon is always collected after the fact and based on the respondents' recall of it. This recollection may not be as accurate as direct measurement or observation. Also, broad surveys make it tedious for respondents to fill out their questions. Moreover, survey researchers must frequently deal with the challenges of low response rates, resulting in lower representativeness of the population [16], a phenomenon that is particularly relevant to our follow-up survey, as it starts from the 653 respondents of the InsighTD survey.

From the InsighTD survey, we seek to identify the practices for TD prevention and the PARs, answering *RQ1* and *RQ3*, respectively. Also, we use the data from this survey to identify TD causes and their relationships with prevention practices, responding to *RQ2*. InsighTD is a globally distributed family of industrial surveys on TD to organize an open and generalizable set of empirical data on the practical problems of TD [36]. As its design supports the survey in different countries, researchers from twelve countries have joined the project. At the time of this writing, the project has concluded data collection for the InsighTD replications in Brazil, Chile, Colombia, Costa Rica, Serbia, and the United States. Results from InsighTD have been disseminated in the technical literature, as shown in Table 1.

To know the level of importance of prevention practices (*RQ1.1*), relationships between practices and causes (*RQ2.1*), and PARs (*RQ3.1*), we designed and performed a second, reduced-scope survey based on results obtained from the InsighTD survey. Asking the participants to rank all PARs, prevention-related practices, and relations between them would have introduced a heavy load on participants, leading to dropouts and incomplete data. So instead, we focused on particular findings from our analysis of the full InsighTD data set, specifically (1) the ten most-cited prevention-related practices and three most recurring practices per category, resulting in 21 practices, (2) all 23 PARs we identified, and (3) the top 12 statistically significant relationships between TD causes and preventive practices with at least five occurrences. We further explain these practices, PARs, and the relationships in Sections 4.1, 4.6, and 4.4, respectively. The complete list of practices, relationships, and PARs used in the study is available in the complementary material [56].

### 3.3 Data Collection

This section presents our two data collection instruments, the InsighTD and reduced-scope surveys.

*3.3.1 The InsighTD Survey.* Table 4 presents the questions of the InsighTD survey considered in this work. This subset allows us to capture the characterization of the participants and their workspace (Q1–Q8). We also included the participants' expression of the TD concept (Q10) and their examples (Q13). Using this example as context, participants reported their experience with TD causes (Q16–Q18) and prevention (Q22–Q23). The complete survey is available in the complementary material [56].

The survey was publicly accessible from mid-2018 until the end of 2019. Invitations were sent only to software practitioners, through LinkedIn, industry-affiliated member groups, mailing lists, and industry partners. In LinkedIn, each InsighTD replication team used specified keywords to search for professionals with specific expertise. For example, we used tester, test analyst, and test manager to find professionals who could provide insightful answers about test debt. Other keywords to each type of debt are presented in Rios et al. [36]. Regarding industry-affiliated member groups,

mailing lists, and industry partners, each InsighTD replication team contacted its industry partners to invite them to participate in the survey.

All collected answers were validated following three acceptance criteria:

— *The participants should consider a TD perspective to answer the questions.* We analyzed the definition of TD described by the participants in Q10 to determine if it was in conformance with the definition[1] of TD used in the InsighTD project [36]. For example, answers like "TD would be like bugs in development code" and "I don't know" were discarded because they do not represent a valid TD definition. On the other hand, answers like "Taking shortcuts now that if not addressed soon will cost much more to resolve in the future" and "Doing something sub optimally now to do it more quickly" passed to the next validation step.

— *The participants should provide a valid example of a TD item.* We analyzed the example of TD described by the participants in Q13. If the example was in conformance with the definition of TD used in the InsighTD project [36], then we concluded that the participant answered the other survey's questions from a valid TD perspective. For example, answers like "I don't know what TD means" and "Errors that arise while fixing other issues" were discarded because they do not represent valid TD examples. On the contrary, answers from participants who provided examples of TD items like "Tests planned but not executed" and "Hard maintenance and future change due to poor documentation from the development team" passed to the next validation step.

— *The participants should provide valid answers in questions on TD causes and prevention.* We analyzed all answers given for Q16 thru Q18, and Q23 to verify whether we can identify causes or preventive practices, respectively. As we did not find any invalid answer, we concluded that the participants did not misunderstand these questions. For instance, we considered the answers given by a participant who reported that "small team, huge system, team turnover…" were the causes of TD.

*3.3.2 Reduced-Scope Survey.* Table 5 presents the questions of the reduced-scope survey. It was composed of three sections: TD prevention-related practices (Q1 thru Q21), relationships between TD prevention-related practices (Q22 thru Q33), and PARs (Q34 thru Q56). In each question, we provided a brief description and introduced the questions related to the elements (practice, PAR, or relation). We presented definitions and descriptive quotes extracted from the InsighTD data. By using a ten-point scale from "not important" to "critical," participants analyzed each element and chose the best option that represents their opinion. Figure 1 shows an example of these questions. The complete survey is available in the complementary material [56].

As we only invited practitioners who had answered the InsighTD survey, we only needed to ask participants to provide their email address, which we could then use as a participant identifier. This allowed us to match the answers collected in the reduced-scope survey to the ones collected in the InsighTD survey.

Before deploying the survey, we conducted three internal revisions, i.e., researchers from the InsighTD project checked the survey questions and structure, suggesting some improvements. After applying the improvements, we piloted the survey with three participants and captured their

[1] The TD definition used in *InsighTD* project is adapted from McConnell [57]: "TD contextualizes the problem of outstanding software development tasks (for example, tests planned but not executed, pending code refactoring, pending documentation update, use of bad design practices, code that does not exhibit good coding practices) as a kind of debt that brings a short-term benefit to the project (normally in terms of higher productivity or shorter release time of software versions), that may have to be paid later in the development process with interest (for example, a poorly designed class tends to be more difficult and costly to maintain than if it had been implemented good object-oriented practices)."

Table 4. Subset of the InsighTD Survey's Questions Related to TD Causes and TD Prevention

| No. | Question (Q) description | Type |
|---|---|---|
| Q1 | what is the size of your company? | Closed |
| Q2 | In which country you are currently working? | Closed |
| Q3 | What is the size of the system being developed in that project? | Closed |
| Q4 | What is the total number of people of this project? | Closed |
| Q5 | What is the age of this system up to now? | Closed |
| Q6 | To which project role are you assigned in this project? | Closed |
| Q7 | How do you rate your experience in this role? | Closed |
| Q8 | Which of the following most closely describes the development process model you follow on this project? | Closed |
| Q10 | In your words, how would you define TD? | Open |
| Q13 | Give an example of TD that had a significant impact on the project that you have chosen to tell us about: | Open |
| Q16 | What was the cause of the example of TD you just described? | Open |
| Q17 | What other cause contributed to the cause you described? | Open |
| Q18 | What other causes contributed to the occurrence of the TD? | Open |
| Q22 | Do you think it would be possible to prevent the type of debt you described in question 13? | Closed |
| Q23 | If yes, how? If not, why? | Open |

Adapted from Rios et al. [36].

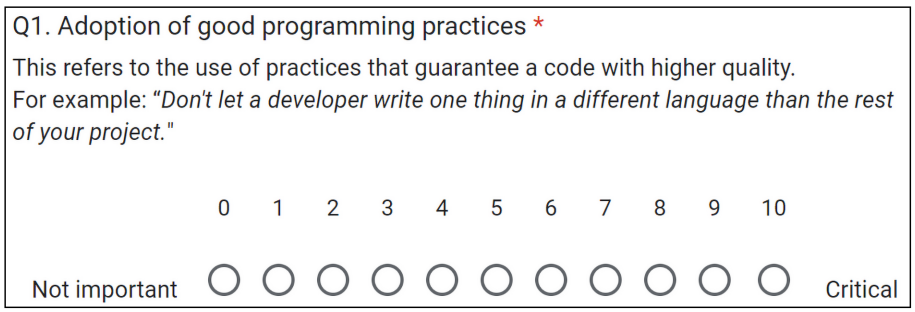



Fig. 1. A screenshot of the survey question for the adoption of good programming practices practice.

opinions on the survey structure and questions, allowing us to calibrate and define the response time. Finally, we analyzed the feedback received in the pilot and applied it to the survey, resulting in its final version.

### 3.4 Data Analysis

This section presents the data analysis procedures we applied to answers collected by the InsighTD and reduced-scope surveys.

*3.4.1 The InsighTD Survey.* We used descriptive statistics for analyzing the closed questions (Q1–Q8). By calculating the mode and median, we have the central tendency of the ordinal and interval data; while counting the share of participants choosing each option available in each closed question, we have information on the nominal data. For the open-ended questions about TD causes and prevention, we applied open coding [2, 10].

Table 5. Example Questions on the Reduced-Scope Survey

| Section | No. | Question (Q) description | Type |
|---|---|---|---|
| 1: TD Prevention-related Practices | - | "A prevention practice is an intentional activity, aligned with the project management plan, that ensures the future performance of the project work" [42]. When applied to TD, a prevention-related practice can support the development team in avoiding the occurrence of debt or increase the teams' capability for executing TD prevention activities.<br><br>Below, there are 21 TD Prevention-related Practices that can be used to avoid debt items or increase the team capability for TD prevention. These practices are based on answers given by software practitioners. For each of them, please indicate how decisive a practice is for supporting TD prevention, considering a scale of 0 to 10, with 0 being "not important" and 10 being "critical." | - |
| | Q1 | Adoption of good programming practices This refers to the use of practices that guarantee a code with higher quality. For example: "Don't let a developer write one thing in a different language than the rest of your project." | Closed |
| | Q2–Q21 | Other Practices | Closed |
| 2: Relationships between prevention practices and causes of TD | - | We also identified a relationship between the TD prevention practices and causes of TD. A cause of TD is a factor that leads software teams to incur debt items in their projects.<br><br>A relationship between a practice and a cause (practice − > cause) indicates that using this practice can increase the possibility of avoiding TD items caused by that cause.<br><br>Below, you will find 12 relationships between Prevention-related Practices and Causes of TD (practice − > cause). For each of them, please indicate how that prevention practice would be decisive to prevent a debt item incurred due to that cause. Please, consider a scale of 0 to 10, with 0 being "not important" and 10 being "critical." | - |
| | Q22 | Adoption of good programming practices − > Sloppy code (practice) Adoption of good programming practices: this means the use of practices that guarantee a code with higher quality. For example: "Don't let a developer write one thing in a different language than the rest of your project." (cause) Sloppy code: refers to code that is written without considering its quality. For example: "Poorly developed code." | Closed |
| | Q23–Q33 | Other relations between practices and causes | Closed |
| 3: PARs for TD non-prevention | - | Unlike the previous sections, now we will address the reasons for failing to prevent TD items. These reasons are called PARs. A PAR explains why a TD item cannot be prevented in a software project.<br><br>Below, there are 23 PARs for TD prevention. For each of them, please indicate how decisive a PAR is for justifying the failure to prevent TD, considering a scale of 0 to 10, with 0 being "not important" and 10 being "critical." | - |
| | Q34 | Architectural evolution This is associated with the constant change in the software architecture. For example: "Due to the constant evolution of architecture." | Closed |
| | Q35–Q56 | Other PARs | Closed |

*Identifying Preventive Practices and PARs (RQ1 and RQ3).* We coded the answers given to Q23 (TD prevention), when Q22 had received a positive answer, thus identifying *prevention practices* (*RQ1*); otherwise, we identified *PARs* (*RQ3*). The open coding was performed by at least three researchers from each InsighTD replication team. Two researchers acted as coders and one as a reviewer. Initially, the coders independently coded the set of all answers for two subsets: one represented the preventive actions, formed by the answers to Q23 when the participants' responses were positive for Q22, and another was composed of the answers to Q23 when the participants' responses were negative for Q22, revealing PARs for failure of TD prevention. We calculated the agreement level among the coders using Cohen's Kappa coefficient [29]. It results in a number $\kappa$ between [−1, +1], indicating the highest disagreement and agreement, respectively. As the threshold cut-off for deciding on the high agreement varies according to the field [29], we considered $\kappa \geq 0.79$ as a high agreement rate, as used in software engineering studies [25]. We obtained a $\kappa = +0.822$, indicating a high agreement rate and reliability for the coding process performed for all InsighTD replication teams.

Coders then had a consensus meeting, and the remaining disagreements were resolved by the reviewer who was an experienced researcher part of each InsighTD replication team. In cases of disagreement, the reviewer coded the answer and compared their results with those reported by the coders. If the reviewer's answer was the same as one of the coders, consensus was reached. Otherwise, the reviewer and coders met to achieve consensus. For example, a participant gave the following answer when describing a preventive practice: "At product conception and in later phases when requesting improvements from the customer, well-defined documentation and an ER model of the solution would perhaps have improved communication between the solution creators and the team that took over the improvements (since the creators were no longer with the company and able to respond). An effort by the team that took on the problem to better understand the solution would also have been welcome." A coder annotated *well-defined ER model*, *communication plan*, and *readiness of the team*, and another one listed *planning*, *good communication between stakeholders*, and *good group work*. As the coders did not have a consensus, the reviewer analyzed the answer and jointly with the coders defined the final codes: *well-defined ER model*, *good communication between stakeholders*, and *team commitment*.

To exemplify the entire process, let us consider the following answers given by three practitioners in Q23 when Q22 had a positive answer: "more structured requirements…" and "better product functional needs forecasting…" As these answers are associated with improvements in requirements, they were unified under the preventive practice *well-defined requirements*. Another example is considering the following answers given by two participants in Q23 when Q22 received a negative answer: "because we always need to make tradeoffs due to time pressure" and "a lot of the TD I've encountered will not be solved because it's a symptom of market forces. There will always be a rush to get a product out." By analyzing these answers, we noticed that they are related to lack of time to perform a TD prevention task; then, they were unified under the PAR *short deadline*.

Finally, we analyzed the extracted codes in both subsets, identifying the codes that had the same meaning. This process resulted in the final list of a set of standard codes. This process was performed iteratively until reaching the state of saturation, i.e., a point where no new codes or categories were identified. We updated the list of preventive practices and PARs presented in our previous work [50] by including data from four new InsighTD replications, totaling 546 participants (including the 182 from the previous work).

The first list of preventive practices and PARs was done by the Brazilian replication team and was sent to the other replication teams to standardize the used nomenclature. The Brazilian replication team also verified the consistency of all lists received from the other replication teams

and consolidated them, resulting in the final list of preventive practices and PARs, along with their number of occurrences.

*Identifying the Relations between Preventive Practices and Causes of TD (RQ2).* For answering RQ2, we used the results obtained from RQ1 and the updated list of causes reported in Ramač et al. [47], which was generated using the same data set used in this work. However, we filtered the answers to only those who provided at least one cause and one preventive practice related to the example provided in Q13, resulting in 538 answers for analysis.

As the participants used the example of TD described in Q13 as context for answering questions on causes (Q16–Q18) and prevention (Q23), we can investigate the co-occurrence of causes and preventive practices. For example, in Q13, a participant reported the following TD item: "the lack of testing caused me to refactor the code more than necessary until it was 100% functional in production." Considering this TD item, the same participant reported the following cause in Q17 "not knowing how to write test classes in the language I was using," and we coded it as *lack of technical knowledge*. Also, the participant suggested how preventing that specific TD item: "studying about tests in the language I was using and testing the classes that needed tests." We coded this answer in two preventive practices: *training* and *creating tests*. Lastly, we identified two relationships between the causes and preventive practices: (1) *lack of technical knowledge* and *training* and (2) *lack of technical knowledge* and *creating tests*. This relationship indicates the co-occurrence between them, i.e., in the presence of a cause (*lack of technical knowledge*), practitioners commonly use a TD preventive practice (*training*). This procedure resulted in a list containing the relationships between causes and preventive practices along with their respective number of occurrences.

As the data does not allow the definition of causal relationships, we only investigated the co-occurrence between them, applying RR (relative risk) to identify statistically significant relationships. RR refers to the likelihood of the occurrence of an event after exposure to a risk variable as compared with the likelihood of its occurrence in a control or reference group [14]. For example, by applying RR, it is possible to verify whether a specific drug can produce a specific side effect analyzing data from two groups [14], one that received the drug and another that receives a placebo. In the context of this work, RR is applied to verify whether a preventive practice can increase or not the possibility of curbing a cause of TD.

RR is calculated as the **absolute risk (AR)** with the risk variable ($AR_{RV}$) divided by the AR in the control group ($AR_{CG}$). Thus, AR represents the number of people who experience an event divided by the number of people exposed to the risk of that event. In our case, we considered the number of occurrences of a relationship between a cause and a preventive practice as the "number of people who experience an event" while the number of occurrences in which a relationship between a cause and a preventive practice did not occur as the "number of people exposed to the risk of that event."

To illustrate how we calculate RR, let us consider the example presented in Table 6. We show the number of occurrences between the cause *lack of team communication* and the preventive practice *good communication between stakeholders*. We found five occurrences of the pair, while in 22 occurrences we identified the cause associated with other preventive practices. For calculating the $AR_{RV}$, we consider the first line of the table. Thus, we have $AR_{RV} = 5/(5 + 22) * 100 = 18.51\%$. We use the second line to calculate $AR_{CG} = 14/(14 + 497) * 100 = 2.74\%$. As RR is a result of $AR_{RV}/AR_{CG}$, we have $RR = 18.51/2.74 = 6.76$. We can interpret this RR value as: using the practice *good communication between stakeholders* can increase the possibility of avoiding TD items caused by *lack of team communication* by almost seven times.

In clinical studies, the RR value calls attention if it is less than 0.5 or more than 2.0, i.e., if the risk is halved or doubled [14]. We used these intervals to measure the effect size of RR value. Thus, if

Table 6. Example of RR Calculation

| Cause<br>Lack of team communication | Preventive practice<br>Good communication between stakeholders | |
|---|---|---|
| | Yes | No |
| Yes | 5 | 22 |
| No | 14 | 497 |

the RR value is more than 2.0, we considered it has *high effect size*, while if it is less than 0.5, we considered it has *low effect size*.

Finally, the RR values are accompanied by **confidence intervals (CIs)**. We calculated these intervals with 95% of confidence, resulting in a $p$-value equals to 0.05. Considering the value and its intervals, we can define whether they are statistically significant following the criteria [14]:

—if RR and its CIs includes the value 1.0, the RR is not statistically significant, indicating that the event can or cannot occur.
—if RR and its CIs are both greater than 1.0, the RR is statistically significant, indicating that the risk is increased in the exposed sample.
—if RR and its CIs are both less than 1.0, the RR is statistically significant, indicating that the risk is reduced in the exposed sample.

*3.4.2 Reduced-Scope Survey.* Using the email address provided for each participant, we were able to match each participant's response to the reduced-scope survey with their demographic data collected in the InsighTD survey. This allowed us to omit characterization questions in the reduced-scope survey, reducing the survey size. We used descriptive statistics to analyze the questions (Q1–Q56) by counting the share of participants choosing each option available in each question to rank the TD prevention-related practices, relations between practices and causes, and PARs.

## 4 Results

This section presents the results we obtained from the InsighTD and reduced-scope surveys.

### 4.1 Demographics

This section presents the characterization of respondents to both the InsighTD and the reduced-scope surveys.

*4.1.1 The InsighTD Survey.* In total, we sent 3,315 invitations for software practitioners and received 745 answers, representing a 22.5% response rate. However, 12% of these answers were discarded as they did not fit the validation criteria previously explained in Section 3.3.1. Thus, we have 653 valid answers from the survey's participants (Brazil: 107 answers, Chile: 89, Colombia: 134, Costa Rica: 145, Serbia: 79, and the United States: 99). The raw data is available in the complementary material [56]. Figure 2 presents the characterization of the participants of the InsighTD project and their work contexts. With respect to most characterization variables, the data represents the entire spectrum of values. We can notice that half of the participants are developers and most are competent or better in their level of experience. They work most often in medium-sized organizations and in teams with 5–9 people. Nearly half follow a hybrid process model. Regarding the systems, the most common age was 2–5 years and the most common system size was between 10KLOC and 1 MLOC.

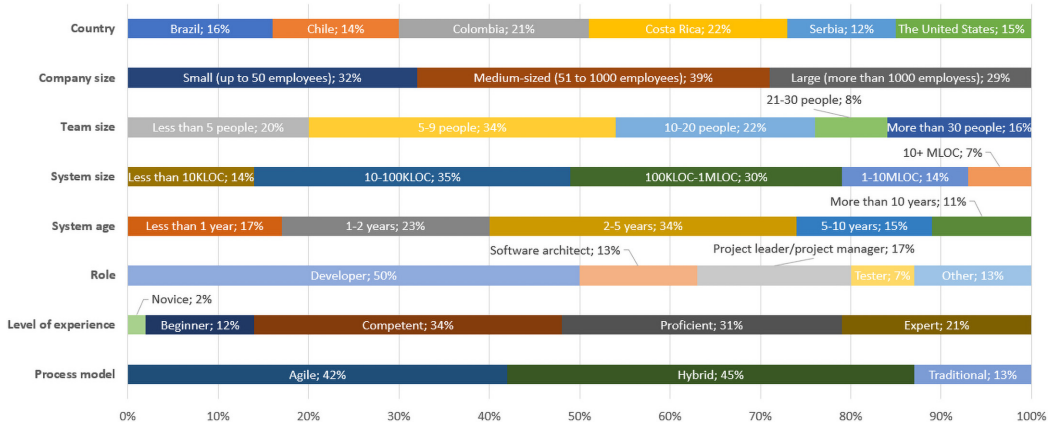



Fig. 2. Participant's demographic from the InsighTD survey.

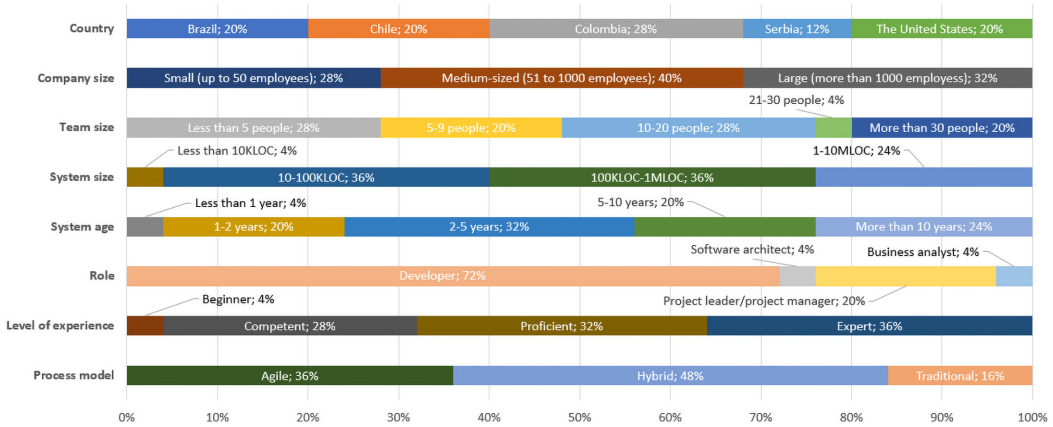



Fig. 3. Participant's demographic from the reduced-scope survey.

In summary, the data set is composed of answers given by practitioners with a variety of roles and levels of experience, and from a variety of development contexts, i.e., with different organization and team sizes, system ages and sizes, and process models.

*4.1.2 Reduced-Scope Survey.* In total, we invited 467 InsighTD respondents who provided their email address in the InsighTD survey. However, only 25 of them answered the reduced-scope survey, representing a 5% response rate. Figure 3 depicts the characterization of the reduced-scope survey respondents. Although these distributions are very close to the larger data set used in this article, the reduced-scope survey respondents worked more often in slightly larger teams (10–20 people as compared to 5–9 in the InsighTD data set) and were more likely to be experts in terms of experience.

## 4.2 RQ1: What Are the Leading Prevention Practices That Can Be Used to Avoid TD?

In Q22 of the InsighTD survey, ~91% of the participants indicated that the TD item described by them in Q13 could be prevented and ~92% of those described how TD could be prevented in

Table 7. Top 10 TD Preventive-Related Practices

| No | Prevention-related practice | #CP[a] | %PP[b] |
|---|---|---|---|
| 1st | Well-defined requirements | 57 | 10% |
| 2nd | Adoption of good programming practices | 49 | 9% |
| 3rd | Better project management | 43 | 8% |
| 4th | Training | 36 | 7% |
| 5th | Following the project planning | 34 | 6% |
| 6th | Improving software development process | 33 | 6% |
| 7th | Improving documentation | 26 | 5% |
| 8th | Using good design practices | 26 | 5% |
| 9th | Well planned deadlines | 26 | 5% |
| 10th | Better project planning | 24 | 5% |

[a]Count of prevention practices.
[b]Percentage of CP in relation to the total of all projects (546).

Q23. For answering RQ1 and its sub-questions, we used this subset of responses, comprised of 546 answers.

We found 89 prevention-related practices that could be used for curbing the presence of TD. Table 7 presents the 10 most commonly cited. This table reports the prevention-related practice name and the total number (i.e., count) of citations (#CP), i.e., the number of projects in which the practice could be used (since each response corresponded to one particular project). The column %PP presents the percentage of #CP in relation to the total of projects (546) that cited at least one preventive practice. It reveals how frequently each preventive practice could be used in software projects. The complete list of TD prevention-related practices along with their definitions and examples of citation is available in the complementary material [56].

In the context of this work, the most-cited prevention-related practice *well-defined requirements* is related to a good definition of the project requirements, ensuring a better understanding of the functionality that will be developed, as described in the quote extracted from the participants' answers: "more structured requirements..." and "better product functional needs forecasting." This practice has not been identified yet in the related work[2] discussed in Section 2.2. The *adoption of good programming practices* practice means the use of practices that guarantee code with higher quality, as described as follows: "don't let a developer write one thing in a different language than the rest of your project" and "better code quality following design principles and patterns." The works conducted by Yli-Huumo et al. [23], Yli-Huumo et al. [24], Rios et al. [37], Silva et al. [59], Apa et al. [12, 13] also identified this practice, but using other nomenclature, such as, *coding standards and guides*, *comment the code*, and *create tutorials on how to fill in the documentation.*

The *better project management* practice is related to the activities performed by the management team, for example, "better project management" and "better organization by management team." This practice has not been identified yet in the related work. The *training* practice indicates that the development team could be trained in the technologies used in the project or training could be offered to level the team, as indicated in the following answers: "keeping the team updated on good development practices" and "considering the study of the technologies used in the project." Codabux et al. [64] and Rios et al. [37] also identified this practice.

[2] A complete comparison between practices identified in this work and those reported in related work is available in Section 5.2.

The *following the project planning* practice indicates that the planning of project activities can prevent the occurrence of TD items, as we can observe in "following the plan that has been defined…" The *improving software development process* practice means that software teams can apply improvements on their development process to prevent TD items, as we can see in "better collaborative processes that also incrementally address quality…" and "use a shorter and iterative development cycle where milestones are clearly laid out and are achievable in a viable time frame." Both practices (*following the project planning* and *improving software development process*) have not been identified yet in related work.

The practices *improving documentation*, *using good design practices*, *well-planned deadlines*, and *better project planning* were cited by 5% of the participants. The *improving documentation* practice indicates that the debt can be prevented when the software team makes the documentation available and updated, as described as follows: "keeping thorough documentation and making it readily available to everyone" and "spend more time on documenting." Only the work performed by Rios et al. [37] also identified this practice, such as, *define process and good practices for documentation*, *have a documentation repository*, and *improve commitment of the team concerning documentation*.

The *using good design practices* practice is related to use of practices to guarantee a design with quality, for example, "by spending more time on design and creating specification" and "good analysis and design must be carried out." The *well-planned deadlines* practice means that the team needs to understand the activities that will be done to estimate the time required by the team, as we can observe in: "adequate time given to developers to read and understand said requirements" and "more time to complete task proper way." Lastly, the *better project planning* practice indicates improving the planning of the project considering the estimation of each activity, for example, "planning instead of imagining…" and "better planning and estimates." None of the related work have identified these practices yet.

> *Finding #1*: *Well-defined requirements*, *adoption of good programming practices*, and *better project management* were the most cited as potential prevention-related practices for avoiding TD items in software projects. Amongst them, only the *adoption of good programming practices* has been identified previously in the technical literature.

After identifying the practices, we noticed that the data revealed different dimensions. These kinds of discoveries are expected when coding qualitative data [2]. We identified the following dimensions related to TD prevention concept:

— *Types of TD prevention practices*. From the data, we found that practitioners have used different types of practices, based on their purpose, to support the TD prevention activity. A practice can either be used to directly prevent a TD item (*preventive action*) or it could contribute more generally to TD prevention (*enabling action*). For example, the *well-defined requirements* practice directly avoids TD items, but the *training* practice improves the team's capability for preventing the debt, which might or might not lead to actually preventing debt. Two researchers conducted the coding process, one assigning the type for each practice and the other reviewing the results. The divergences were resolved between those researchers in a consensus meeting.

— *Categories of TD prevention practices*. We also found that many of the practices are related to each other, supporting us to organize them into categories. The *category* dimension divides the practices according to the related software development issues, such as development, management, people, and so on. Similarly to the *type* dimension, one author grouped the

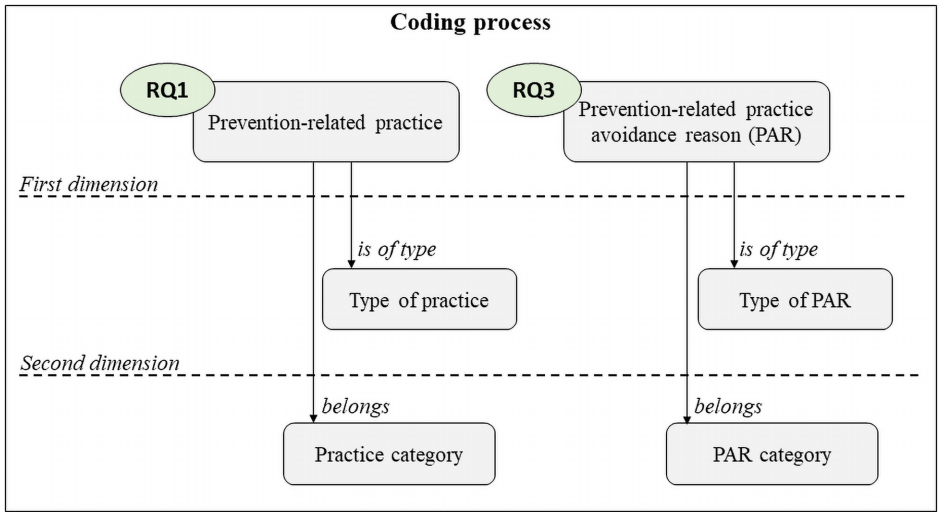



Fig. 4. Data dimensions arising from coding process and their relationship with RQs.

Table 8. Types of TD Prevention-Related Practices

| Type of prevention-related practice | #P[a] | #CP[b] | %PP[c] |
|---|---|---|---|
| Preventive action | 46 | 517 | 95% |
| Enabling TD prevention | 44 | 303 | 55% |

[a]Count of unique cited prevention-related practices.
[b]Count of prevention-related practices.
[c]Percentage of CP in relation to the total of all projects (546).

practices into categories, while another reviewed the grouping. The divergences were resolved between those researchers in a consensus meeting. To name the categories, we used the categories proposed by Rios et al. [36] and presented in the complementary material [56]. For example, the preventive practices *well-defined requirements* and *using good design practices* were grouped in the *internal quality issues* category.

Figure 4 summarizes the coding process we performed to investigate the dimensions of our data revealed during our analysis, as we described above. It shows that we identified TD prevention practices and PARs (further discussed in Section 4.6) by conducting open coding and grouping them (practices and PARs) into different properties and dimensions [2], i.e., types and categories.

Table 8 presents the two types of prevention practices, reporting each type's name, the number of unique prevention practices (#P), and the total number (i.e., count) of prevention practices (#CP) cited in each type. #CP also indicates the number of projects that could use that preventive practice in each prevention practice type. Column %PP corresponds to the percentage of #CP in relation to the total of all projects, indicating how frequently each prevention practice type could be used in software projects. Although there are almost the same number of preventive practices in each type, the quantity of citations (#CP) and frequency of use (%PP) are very different. While preventive actions were more cited and could be used in almost all projects, enabling TD prevention could be used in just over half of the projects. We also found that 96 participants identified both prevention practice types in their answers. In other words, about 18% of the participants could use preventive actions and enabling TD prevention practices together to avoid TD items from their projects.

Table 9. Comparison of Our Practices with Ones Reported by Codabux et al. [64]

| Type | Prevention-related practice from | |
|---|---|---|
| | Our study | Codabux et al. [64] |
| Preventive actions | Code review | Pair programming |
| | Continuous integration | Continuous integration |
| | Following well-defined project process | Conformance to process and standards |
| | Refactoring | Refactoring |
| | Use the most appropriate version of the technology | Tools |
| Enabling TD prevention practices | Good communication between stakeholders | Customer feedback |
| | Implementation of a TD payment strategy | Having each development team dedicating one iteration during a set release period towards debt reduction |
| | Improve schedule to include tests and bug fixing | Bugs bashes |
| | Prioritization of TD payment | Having dedicated teams whose primary focus is on reducing TD |
| | Training | Education and training |
| | - | Allowing slack |
| | - | Reflective improvement |

Codabux et al. [64] also identified a variety of prevention practices. Table 9 presents the comparison between their practices and our identified practices. We can notice that only the practices *allowing slack* and *reflective improvement* from Codabux et al. [64] were not confirmed by our results.

> *Finding #2*: When practitioners talk about TD prevention, they report *preventive actions* as well as *enabling TD prevention* practices. Preventive actions are more often cited by the survey's respondents.

The other dimension we identified, *category*, allows us to divide the practices into groups representing software development concerns. As a result, this dimension has the following values:

- *Development issues*: it groups nine practices associated with software development activities, such as *adoption of good programming practices*, *appropriate reusing of code*, and *considering technical constraints*.
- *Infrastructure*: it encompasses two practices (*organizing code repository* and *use the most appropriate version of the technology*) associated with related to tools, technologies, and development infrastructure.
- *Internal quality issues*: it has six practices that can be employed to address limitations that compromise the internal quality of the software. For example, *improving the maintainability of the project*, *refactoring*, and *using good design practices*.
- *Methodology*: it refers to practices related to the process followed by the team. This category has 28 practices, such as, *improving software development process*, *improving documentation*, and *creating tests*.
- *Organizational*: it groups ten practices associated with organizational decisions. For example, *better understanding of development process by businesses*, *contracting a domain expert to architect the project*, and *technical support*.
- *People*: it includes nine practices related to team characteristics. *Technical knowledge*, *organized team*, and *discipline* are example of practices composing this category.

Table 10. Categories of Prevention-Related Practices

| Category of prevention-related practices | #P[a] | #CP[b] | %PP[c] |
|---|---|---|---|
| Planning and management | 25 | 287 | 53% |
| Methodology | 28 | 214 | 39% |
| Internal quality issues | 6 | 122 | 22% |
| Development issues | 9 | 87 | 16% |
| Organizational | 10 | 56 | 10% |
| People | 9 | 46 | 8% |
| Infrastructure | 2 | 8 | 1% |

[a]Count of unique cited prevention practices.
[b]Count of prevention practices.
[c]Percentage of CP in relation to the total of all projects (546).

—*Planning and management*: encompasses 25 practices related to managerial activities. For example, *better project management*, *following the project planning*, and *allocation of qualified professionals*.

Table 10 presents the categories of prevention practices, reporting the category's name, the number of unique practices cited (#P) and the total number (i.e., count) of practices (#CP) cited in each category. #CP also indicates the number of projects that could use a practice in each category. Lastly, the column %PP corresponds to the percentage of #CP in relation to the total of all projects.

The *planning and management* categories have the greatest number of practices, followed by *methodology*. This implies that managerial and methodological practices are crucial for TD prevention. Also of note are the *internal quality issues* and *development issues* categories, which are cited by 22% and 16% of the projects, respectively. This result indicates that practices related to the quality of the system and the code implemented by the team also matter for TD prevention.

We also investigated the relationship between categories and types of prevention practices, as shown in Figure 5. The categories *planning and management*, *people*, *methodology*, *infrastructure*, and *development issues* have both types of prevention practices. The categories *planning and management* and *people* encompass more enabling TD prevention practices than preventive actions. The contrary occurs for the categories *methodology* and *development issues*. The *organizational* category comprises only enabling TD prevention practices, while *internal quality issues* has only prevention actions. The *infrastructure* category contains one of each type of prevention practice. This can be interpreted to mean that managerial, organizational, and people related practices can support TD prevention initiatives, but actual TD prevention happens through practices associated with the internal quality of the system, and the methodology and development process followed by the team.

*Finding #3*: The prevention practices were organized into seven categories. Although the categories *planning and management* and *methodology* were the most cited, the first encompasses more *enabling TD prevention practices* and the latter groups more *prevention actions*.

### 4.3 RQ1.1: How Do Software Practitioners Perceive the Importance of the Prevention Practices?

The answers given to questions Q1 to Q21 of the reduced-scope survey (Table 5), show that all analyzed prevention practices are rated fairly high in importance, with a mean value ranging from

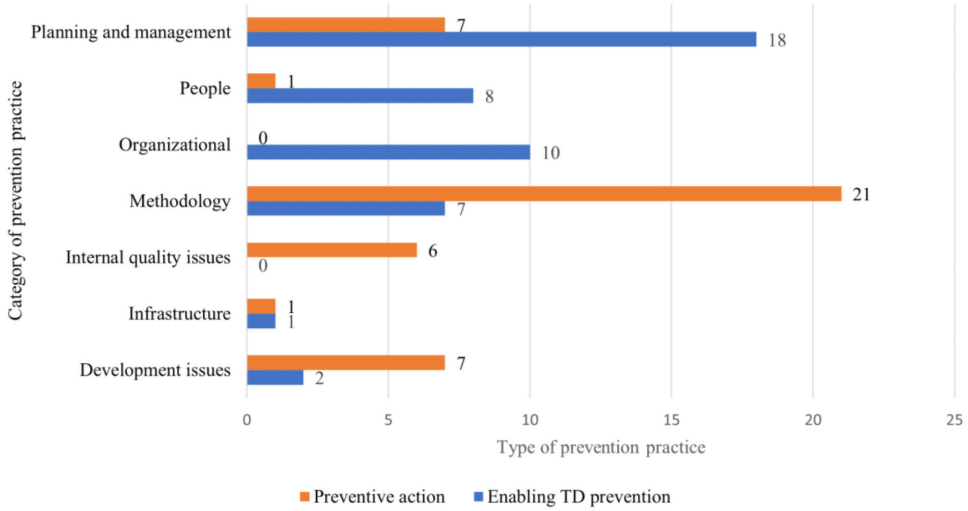



Fig. 5. Relationship between categories and types of TD prevention-related practices.

6.56 to 8.8 out of 10. Figure 6 depicts this information and the interquartile range considering the responses of all participants.

The highest-rated practice is *well-defined architecture* (mean of 8.8), followed by *well-defined requirements* (8.52) and *using good design practices* (8.44). These practices are part of the *internal quality issues* category, demonstrating that practices from this category are more critical to TD prevention initiatives. This result differs from what is presented in Table 10, where practices from *planning and management* and *methodology* are the most cited for TD prevention. However, practices from these categories are still rated as important (*planning and management* category: *better project management* (7.8), *better project planning* (7.64), *well planned deadlines* (7.36), and *follow project planning* (6.56); *methodology* category: *creating tests* (8.36), *improving the software development process* (8.24), and *improving documentation* (7.56)).

From the top ten most cited practices (see Table 7), *well-defined requirements* (8.52), *using good design practices* (8.44), *adoption of good programming practices* (8.4), *improving the software development process* (8.24), and *training* (8.08) have a mean greater than 8.0, indicating that these practices are essential for increasing the capability of software teams in preventing the debt.

> *Finding #4*: *Well-defined architecture*, *well-defined requirements*, and *using good design* practices are rated as the most important for TD prevention. These practices are from the *internal quality issues* category, indicating that investing effort to improve internal software quality is decisive in curbing TD items.

### 4.4 RQ2: How Do Prevention Practices Relate to Causes of TD in Software Projects?

Table 11 presents the relationship between the top 10 TD causes (from Table 2) and the top 10 prevention-related practices, indicating the number of co-occurrences for each combination. We italics the highest number of co-occurrences for each cause.

Except for the practices *better project management* and *improving documentation*, all the other practices composing the top 10 could be used for preventing any of the top 10 causes. TD items

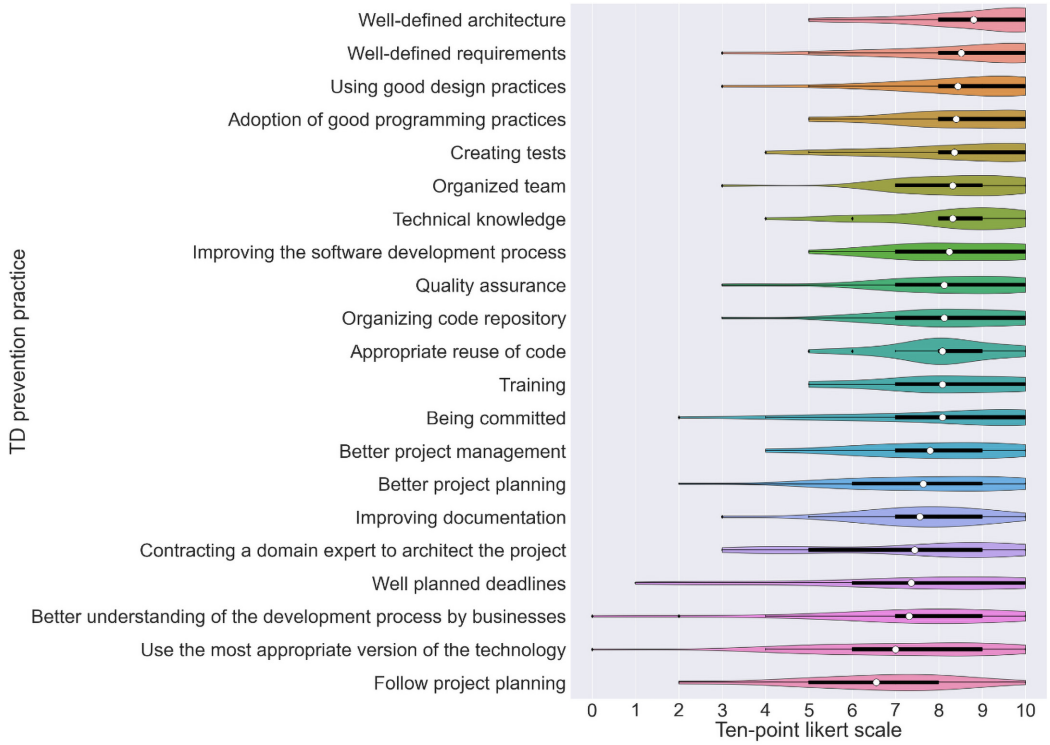



Fig. 6. Violin plots of the distributions of importance given to each practice. The thicker horizontal bar indicates the interquartile range and the white circle indicates the mean.

caused by *deadline* could be prevented using mainly the practices *following the project planning*, *better project management*, and *well-defined requirements*. TD items incurred as a consequence of the causes *not effective project management*, *inappropriate planning*, and *inappropriate/poorly planned/poorly executed test* could be mainly avoided by applying the *better project management* practice. Results also indicate that the *adoption of good programming practices* practice could be used in the presence of the causes *lack of experience* and *non-adoption of good practices*. The *well-defined requirements* practice could be used in the presence of the causes *focus on producing more at the expense of quality* and *lack of qualified professional*. Lastly, TD items caused by *lack of technical knowledge* could be avoided performing the *training* practice, while the *improving software development process* practice could curb items caused by *inaccurate time estimate*.

We also investigated whether the co-occurrence between causes and prevention-related practices are non-random, applying RR. Table 12 presents the statistically significant relationships found between the top 10 TD causes and the top 10 TD preventive practices. The table reports the cause and its corresponding preventive practice, along with the RR value and its CIs. We can notice that the RR values presented in the table are statistically significant because both RR values and their CIs did not include the value 1.0. This is an indication that the groupings of causes and prevention practices are non-random. The complete list is available in the complementary material [56].

The pair composed of the *lack of technical knowledge* cause and the *training* practice has the biggest RR value (4.26 = ~4). This means that the possibility of preventing TD items caused by *lack of technical knowledge* is four times greater when applying *training* than when not applying *training*.

Table 11. Relationship between Top 10 Causes and Top 10 Prevention-Related Practices Along with the Number of Co-Occurrences of Each Combination

| Prevention-related practice | Cause | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Deadline | Not effective project mgmt.. | Inappropriate planning | Lack of technical knowledge | Focus on producing... quality | Inappropriate... test | Lack of experience | Inaccurate time estimate | Lack of qualified professional | Non-adoption of good practices |
| Well-defined requirements | *13* | 8 | 2 | 3 | 7 | 2 | 4 | 4 | *6* | 3 |
| Adoption of good programming practices | 7 | 8 | 4 | 9 | 4 | 4 | *5* | 2 | 3 | *13* |
| Better project management | *12* | *9* | *13* | 3 | 2 | *5* | 4 | 4 | 0 | 2 |
| Training | 5 | 4 | 7 | 13 | 4 | 4 | 4 | 4 | 4 | 8 |
| Following the project planning | *15* | 4 | 7 | 3 | 4 | 1 | 1 | 4 | 4 | 5 |
| Improving software dev. process | 5 | 6 | 4 | 6 | 7 | 1 | 2 | *6* | 4 | 2 |
| Improving documentation | 8 | 5 | 1 | 4 | 4 | 0 | 3 | 0 | 4 | 1 |
| Using good design practices | 8 | 6 | 3 | 5 | 1 | 4 | 3 | 2 | 1 | 3 |
| Well planned deadlines | 11 | 3 | 2 | 2 | 5 | 1 | 1 | 1 | 3 | 1 |
| Better project planning | 7 | 3 | 5 | 2 | 6 | 2 | 2 | 3 | 3 | 1 |

Table 12. Statistically Significant Relationships between Top 10 Causes and top 10 Prevention-Related Practices

| Cause | Prevention-related practice | RR | CI Min 95 | CI Max 95 |
|---|---|---|---|---|
| Lack of technical knowledge | Training | 4.26 | 2.28 | 7.98 |
| Non-adoption of good practices | Adoption of good programming practices | 3.63 | 2.06 | 6.39 |
| Focus on producing more at the expense of quality | Better project planning | 2.87 | 1.18 | 6.98 |
| Non-adoption of good practices | Training | 2.79 | 1.34 | 5.79 |
| Inappropriate planning | Better project management | 2.76 | 1.51 | 5.04 |
| Deadline | Following the project planning | 2.39 | 1.24 | 4.62 |
| Inappropriate planning | Well-defined requirements | 0.24 | 0.06 | 0.95 |

Another point that caught our attention was that some causes are strongly related to more than one practice. For example, the TD items caused by *non-adoption of good practices* can be avoided by *adoption of good programming practices* and by *training*, which is not surprising, but what is interesting is that they have different RR values. While *adoption of good programming practices* increases the chance of preventing this type of TD four times, the practice *training* increases the chance almost three times. The greater the RR value for a specific practice, the more valuable it becomes.

Lastly, only the relationship between the *inappropriate planning* cause and the *well-defined requirements* practice (highlighted in grey) had a low RR, which means that that preventive practice would not be a good choice to prevent TD items stemming from that cause (RR = 0.24).

*Finding #5*: By applying the RR, we concluded that the groupings of causes and prevention practices are non-random, i.e., practitioners have used specified practices to prevent specified causes of debt to increase the possibility of success in their prevention initiatives.

*Finding #6*: By applying the *training* practice, it is possible to increase up to four times the possibility of preventing the *lack of good practices* cause. However, by performing the *well-defined requirements* practice preventing the *inappropriate planning* cause, the chance of avoiding it does not even reach 50%.

### 4.5 RQ2.1: How Do Software Practitioners Perceive the Importance of the Identified Relationships?

Response data for questions Q22 to Q33 of the reduced-scope survey show how decisive a prevention practice is to avoid a debt item incurred due to a cause. As these answers are on a 10-point scale, we calculated a mean, revealing that most of the analyzed relationships between prevention practices and causes have a mean greater than 8.0. This indicates that these relationships are important to guide software practitioners to choose practices for curbing TD items. Figure 7 shows the mean and the interquartile range for each relation.

The *adoption of good programming practices –> sloppy code* relation was the most highly rated (mean of 8.88), followed by *improving software development process –> lack of a well-defined process* (8.72) and *good communication between stakeholders –> lack of team communication* (8.64). This implies that these practices are useful to curb their corresponding causes of TD.

*Better project planning –> focus on producing more at the expense of quality* (7.88), *risk and impact analysis –> inappropriate planning* (7.64) and *following the project planning –> deadline* (7.44) relations had the lowest means, indicating that although they are important, their practices are not so decisive for preventing their correspondence causes.

*Finding #7*: All TD prevention practices we analyzed were considered important for preventing TD originating from the TD causes with which they were associated (see Figure 7). Applying the *adoption of good programming practices* to avoid *sloppy code* is the most pronounced association.

### 4.6 RQ3: What Are the PARs Considered by Software Practitioners for the Failure to Prevent TD?

In the InsighTD survey, in total, 9% of the participants indicated that the TD item described in Q13 could not be prevented and ~80% of those explained in Q23 why the TD could not be prevented. For answering RQ2 and its sub-questions, we used this subset of responses, composed of 49 answers.

We found 23 PARs that could be used by software practitioners for explaining the failure to prevent TD. Table 13 presents the 10 most commonly cited. This table reports the PAR name and the total number (i.e., count) of citations (#CPAR), i.e., the number of projects in which the PAR could be used (since each response corresponded to one particular project). The column %PARP

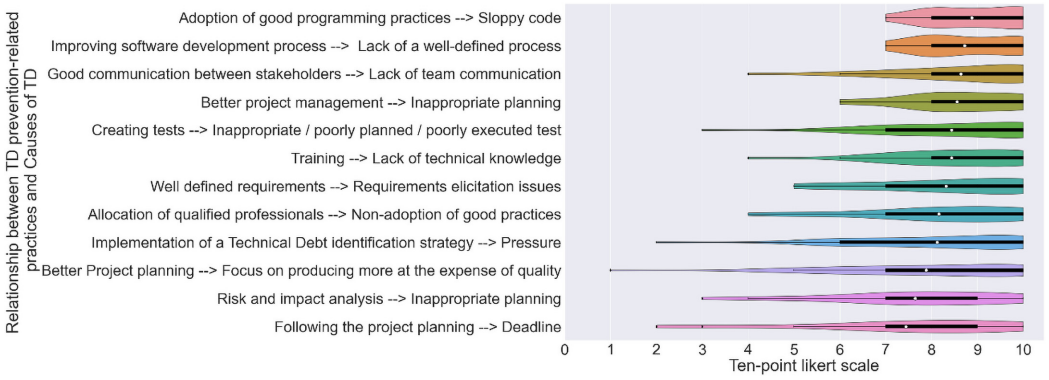



Fig. 7. Violin plots of the distributions of importance given to each relationship between TD prevention-related practice and cause of TD. The thicker horizontal bar indicates the interquartile range and the white circle indicates the mean.

Table 13. Top 10 PARs

| No | PAR | #CPAR[a] | %PARP[b] |
|---|---|---|---|
| 1st | Short deadline | 14 | 29% |
| 2nd | Ineffective management | 7 | 14% |
| 3rd | Lack of predictability in the software development | 5 | 10% |
| 4th | Requirements change | 5 | 10% |
| 5th | Pressure for results | 4 | 8% |
| 6th | Lack of technical knowledge | 3 | 6% |
| 7th | Documentation issues (lack of or non-updated) | 2 | 4% |
| 8th | Lack of concern about maintainability | 2 | 4% |
| 9th | Lack of good technical solutions | 2 | 4% |
| 10th | Lack of process maturity | 2 | 4% |

[a]Count of practice avoidance reason.
[b]Percentage of CPAR in relation to the total of all projects (49).

presents the percentage of #CPAR in relation to the total of projects (49) that cited at least one PAR. It reveals how frequently each PAR could be used in software projects. The complete list of PARs along with their definitions and examples of citation are available in the complementary material [56].

The *short deadline* PAR was the most cited and refers to software teams that do not have time to prevent TD items, as described in the quote extracted from the participants' answers: "because we always need to make tradeoffs due to time pressure" and "a lot of the TD I've encountered will not be solved because it's a symptom of market forces. There will always be a rush to get a product out." The *ineffective management* PAR is related to lack of managerial activities that can support the TD preventive initiatives that could be performed by software teams, as described in "management and process were not mature enough to prevent TD."

The *lack of predictability in the software development* PAR refers to the difficulty in knowing the complexity of software development activities to satisfy all stakeholders, for example, "no large project ever has the foresight to account for every scenario" and "it's difficult to see into the future to know what it will take to develop software." The *requirements change* PAR indicates that changes in requirements can result in refactoring the code or reducing its maintainability, as we can observe in "requirements are always going to change during development..." and "because when the client

asks for features abruptly, no matter how generalized the architecture is towards the problem, with an outlier there may be, that can mean a refactor of the code, and that could dirty the code, reducing its maintainability." These two PARs are related and somewhat similar, but *lack of predictability in the software development* refers to the inability of a team to have the foresight to know that TD prevention activities are needed. On the other hand, *requirements change* refers specifically to the chilling effects of expected requirements volatility on the motivation to prevent TD.

The *pressure for results* PAR means that software teams cannot prevent TD items because they are under too much pressure for results, as indicated in "no, because the pressure of business and priorities simply disrupts the rigorous technical work" and "the pressure for results and compliance schedules (which are always required in terms of monetary benefit), mean that there is TD practically from day 1 of the project." The *lack of technical knowledge* PAR refers to the need to increase the technical knowledge of the software team to avoid the debt, as described as follows: "the solution adopted was one of the best considering the restrictions of available infrastructure, deadline and technical knowledge" and "I believe that it would not be possible to avoid this TD, as the main cause is the lack of knowledge of ReactJS. Also, the type of knowledge needed is very specific and not something you can get through documentation or courses. It is necessary to use ReactJS in practice to solve real problems to understand what the limitations/difficulties of native 'state' control are, and only then look for alternatives."

The *documentation issues (lack of or non-updated)* PAR is related to insufficient or out-of-date software documentation limiting software teams in performing its activities, for example, "when you're building on a platform that does not have sufficient documentation, you may not know how your code will work with the limits until a prototype is tested" and "documentation for the customer must always be up to date." The *lack of concern about maintainability* PAR refers to a lack of concern on the part of developers, as we can observe in "maintainability is seldom a developer's concern." The *lack of good technical solutions* PAR is related to the limited solution choices available to the software team, for example, "at the time the decision was made, we investigated all possible solutions and came up with the 'least bad' solution." The *lack of process maturity* PAR is indicated in quote, such as "management and process were not mature enough to prevent TD."

> *Finding #8*: *Short deadline* and *ineffective management* were the most cited PARs that could be used to explain the failure to prevent TD.

Analyzing the top ten PARs, we noticed that the PARs have different types, similar to TD prevention practices (see Section 4.2). For example, the PARs *short deadline*, *pressure for results*, *lack of technical knowledge*, and *lack of good technical solutions* are PARs that are not within the team's control (an *impediment*). On the other side, *ineffective management*, *lack of predictability in the software development*, *requirements change*, *documentation issues (lack of or non-updated)*, *lack of concern about maintainability*, and *lack of process maturity* are *decision factors* considered by the team in making explicit decisions to not take steps to prevent TD. For identifying the types of PARs, the first author coded the PARs by type, and the last author reviewed the results. Divergences were resolved in a meeting. Therefore, the *type* dimension for PARs groups reasons for failure to prevent TD that are part of a team's decision or are outside of team's control and has the following values: *impediment* and *decision factor*.

Table 14 presents the identified types of PARs, reporting the type's name, the number of unique PARs cited (#PAR), and the total number (i.e., count) of PARs (#CPAR) cited in each type. #CPAR also indicates the number of projects that used that PAR for justifying the failure to prevent TD items in each PAR type. Column %PARP corresponds to the percentage of #CPAR in relation to

Table 14. Types of PARs

| Type of PAR | #PAR[a] | #CPAR[b] | %PARP[c] |
|---|---|---|---|
| Decision factor | 10 | 27 | 55% |
| Impediment | 13 | 34 | 69% |

[a]Count of unique cited PAR.
[b]Count of PARs.
[c]Percentage of CPAR in relation to the total of all projects (49).

the total of all projects, indicating how frequently each PAR type was used in software projects. We found almost the same quantity of PARs in each type, but the quantity of citations is different among them. Impediments are the most common PAR type and can be used in 69% of the projects. Decision factors, in turn, can be used in 55% of the projects.

*Finding #9*: The PARs that explain the failure to prevent TD can be part of a decision of the team itself or an impediment beyond the control of the team. Both types (decision factors and impediments) could be used by more than 50% of the projects.

Taking a look at the top ten PARs, we identified that many of them are related to each other and could be organized into categories related to software development issues. For example, the PARs *debt close to the project end*, *ineffective management*, *lack of concern about maintainability*, and *short deadline* could be grouped into a category named *planning and management*, because they are related to managerial activities performed in a software project. The process for grouping the PARs into categories was conducted by the first author and reviewed by the last author. Divergences were resolved in a meeting. To name our categories, we used the categories defined by Rios et al. [36] and presented in the complementary material [56]. Therefore, the *category* dimension for PARs divides them into groups considering the software development issues, having the following values:

— *Development issues*: groups PARs related to software development activities. This category is composed of five PARs, like *architectural evolution*, *continuous change of coding standards*, *lack of good technical solutions*, *legacy system difficult to heal*, and *requirements change*.
— *External factors*: encompasses three PARs associated with factors that software teams cannot control. *Differences among stakeholders*, *not sure if client will accept it*, and *pressure for results* are the PARs in this category.
— *Lack of knowledge*: has PARs related to the need of technical knowledge. This category is composed of two PARs (*lack of information* and *lack of technical knowledge*).
— *Methodology*: refers to PARs associated with the process followed by the team. *Development teams interdependency*, *documentation issues (lack of or non-updated)*, *lack of predictability in the software development*, and *lack of process maturity* are the PARs in this category.
— *Organizational*: groups PARs related to organizational decisions. This category has the following three PARs: *lack of financial resources*, *lack of qualified professionals*, and *restrictions on available infrastructure*.
— *People*: includes PARs associated with team characteristics. *Lack of experience* and *people issues* are the PARs in this category.
— *Planning and management*: has PARs associated with managerial activities. This category includes four PARs, like *debt close to the project end*, *ineffective management*, *lack of concern about maintainability*, and *short deadline*.

Table 15. Categories of PARs

| Category of PARs for TD non-prevention | #PAR[a] | #CPAR[b] | %PARP[c] |
|---|---|---|---|
| Planning and management | 4 | 24 | 49% |
| Development issues | 5 | 11 | 22% |
| Methodology | 4 | 10 | 20% |
| External factors | 3 | 6 | 12% |
| Lack of knowledge | 2 | 4 | 8% |
| Organizational | 3 | 4 | 8% |
| People | 2 | 2 | 4% |

[a]Count of unique cited PARs for failure to prevent TD.
[b]Count of PARs.
[c]Percentage of CPAR in relation to the total of all projects (49).

Table 15 presents the categories of PARs, reporting the category's name, the number of unique PARs cited (#PAR) and the total number (i.e., count) of PARs (#CPAR) cited in each category. #CPAR also indicates the number of projects that considered a PAR for explaining the failure to prevent TD items in each category. Lastly, the column %PARP corresponds to the percentage of #CPAR in relation to the total of all projects.

The *planning and management* category has the greatest number of PARs, followed by *development issues* and *methodology*. One can expect that *development issues* are decisive for failure to prevent TD, and this category impacted 22% of the projects. However, the categories *planning and management* and *methodology* impacted 49% and 20% of the projects, respectively. Then, we can conclude that software teams should pay more attention to issues related to planning, management, and methodology when they are concerned with failure to prevent TD.

Figure 8 presents the relationship between categories and types of PARs. The categories *planning and management*, *development issues*, and *people* encompass both types of PARs, in nearly equal numbers. This implies that software teams should negotiate with their managers to create a scenario more conducive to TD prevention, but also the teams should examine their own attitudes towards TD prevention, because decisions made by them impact their TD prevention initiatives as well.

The *methodology* category only has decision factors and the categories *organizational*, *lack of knowledge*, and *external factors* groups only impediments. This is expected, because the process model adopted by the team can influence its decision about TD prevention, and other factors out of the team's control can also contribute to the failure to prevent TD.

*Finding #10*: The PARs were organized into seven categories. The categories *planning and management*, *development issues*, and *methodology* were the most cited. While *planning and management* and *development* issues group decision factors and impediments, the *methodology* category only encompasses decision factors.

### 4.7 RQ3.1: How Do Software Practitioners Perceive the Importance of These PARs for Failure to Prevent TD?

Based on the answers given for Q34 to Q56 of the reduced-scope survey, we rated the PARs concerning their importance for failure to prevent TD, as shown in Figure 9. The means range from 5.4 (*continuous change of coding standards* PAR) to 8.64 (*short deadline* PAR) out of 10, revealing that the respondents have a variety of points of view on the analyzed PARs.

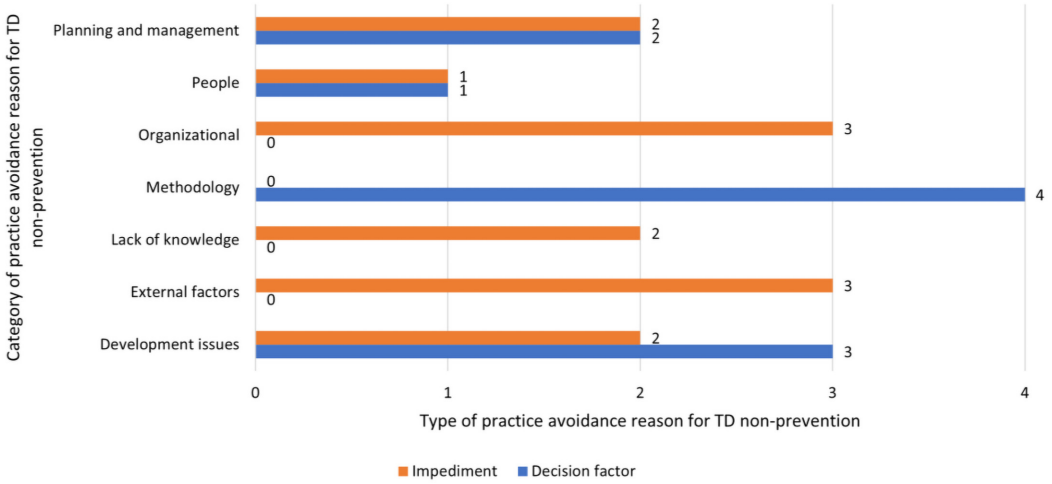



Fig. 8. Relationships between categories and types of PARs.

*Short deadline* (mean of 8.64), *legacy system difficult to heal* (8.36), *pressure for results* (8.16), and *lack of information* (8.04) PARs were rated as most important. From these PARs, only *short deadline* (1st position) and *pressure for results* (5$^{th}$) are also present in the top ten most cited (see Table 13) in the InsighTD data. This implies that they are more critical in explaining the failure to prevent TD.

*Continuous change of coding standards* (5.4) and *architectural evolution* (5.88) were the least important as well as the least cited to explain the failure to prevent TD, indicating that TD prevention initiatives could de-prioritize these PARs when elaborating their strategies for increasing the team's capability for TD prevention.

> *Finding #11*: *Short deadline* PAR is the most mentioned and critical for explaining the failure to prevent TD. Initiatives for increasing the capability of software teams in TD prevention must invest effort in negotiating deadlines considering TD prevention activities in their schedule.

## 5 Discussion

This section discusses the findings per RQ, compares the findings with related and our previous studies, presents the map that organizes the TD prevention-related practices and PARs along with their types and categories, and presents the Sankey diagram that organizes the relationships between TD prevention-related practices and causes of TD.

### 5.1 Revisiting the RQs

Concerning RQ1 (What are the leading prevention practices that can be used to avoid TD?), this study reported 89 prevention-related practices reported by practitioners. The practices *well-defined requirements*, *adoption of good programming practices*, and *better project management* were the most cited, indicating that improvements applied to software requirements, code, and management are necessary to increase the team's ability to prevent debt items. Also, we found that the practices can directly prevent debt items (prevention actions) or support TD prevention initiatives (enabling practices). We categorized all practices into seven categories related to software development

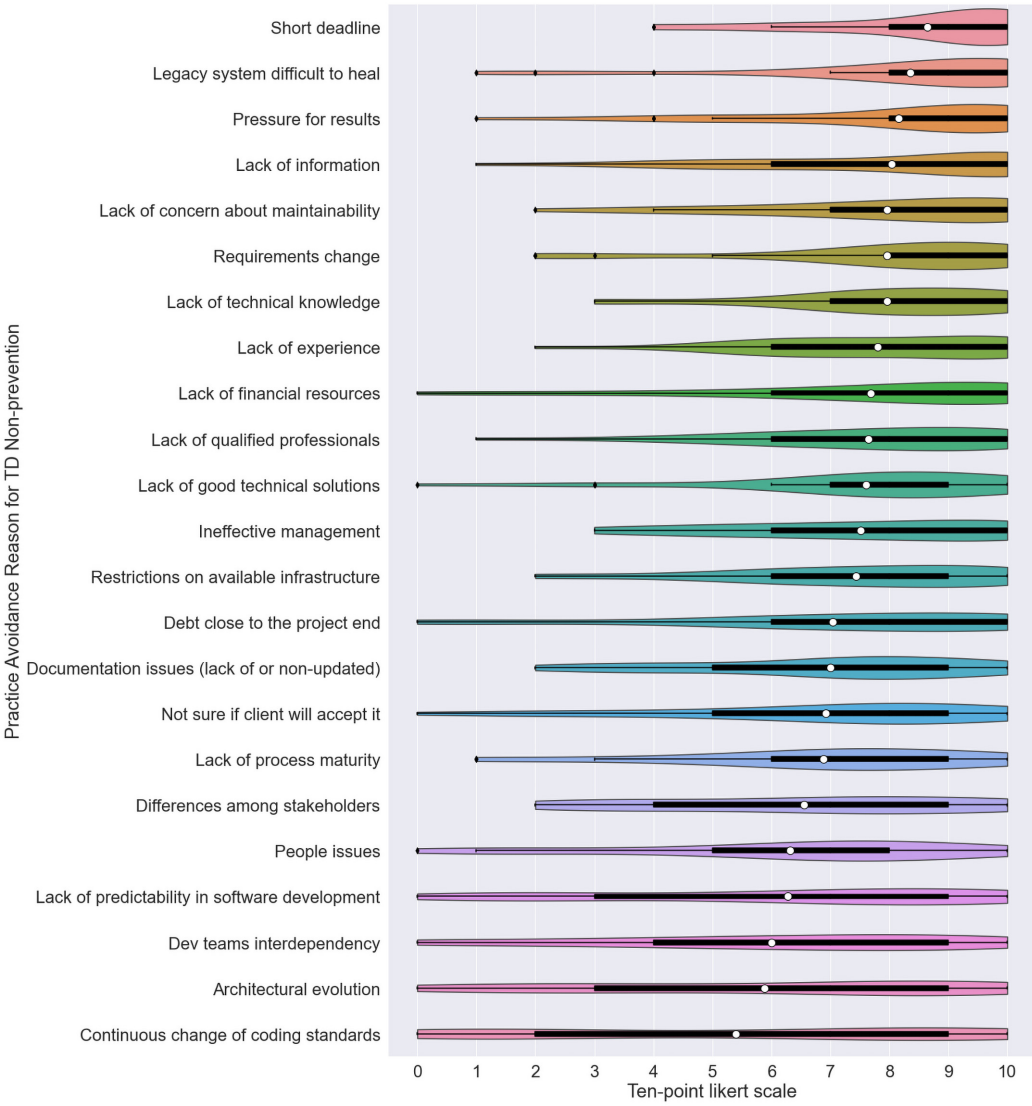



Fig. 9. Violin plots of the distributions of importance given to each PAR for TD non-prevention. The thicker horizontal bar indicates the interquartile range and the white circle indicates the mean.

concerns. The categories *planning and management* and *methodology* stand out, encompassing the greatest number of prevention-related practices.

From the reduced-scope survey, we found that *well-defined architecture* (mean of 8.8), *well-defined requirements* (8.52), and *using good design practices* (8.44) were the most important prevention practices. As these practices are from the *internal quality issues* category, we can conclude that software teams can invest in internal software quality to increase their capability to prevent the debt.

With respect to RQ2 (How do prevention practices relate to causes of TD in software projects?), we found relationships between the preventive practices and the causes of TD. For example, the

practices *well-defined requirements*, *following the project planning*, and *better project management* were indicated to prevent the TD items in the presence of the *deadline* cause. Furthermore, using the RR, we found that some of those relationships are statistically significant, and we calculated how much the use of a preventive practice can increase the possibility for preventing TD in the presence of specific TD causes. For example, when a software team applies the *training* practice in their project, it can increase by almost four times the possibility to prevent TD caused by *lack of technical knowledge*.

It is worth saying that, although some of the findings do not seem to be surprising (e.g., using *training* to avoid the TD due to the *lack of technical knowledge* cause), they were gathered through a well-grounded empirical study, revealing or confirming issues that technical literature or experience may have implied before. Further, some of these relationships may not be so obvious on closer examination. For example, while it seems self-evident that *training* will mitigate *lack of technical knowledge*, what our results show is that this mitigation occurs in such a way as to actually prevent TD items, and by how much. This allows informed choices to be made about where to apply resources towards implementing these preventive practices. It can also allow teams to tailor these practices, knowing which TD causes they are likely to affect. This targeted understanding can, in the end, assist software practitioners in improving their capability to prevent TD items.

RQ2 also addressed the importance of the relationships we found between TD prevention practices and TD causes, according to respondents of our reduced-scope survey. Our data showed that the most important relationships (scored on a scale of 10) were *Adoption of good programming practices –> sloppy code* (mean of 8.88), *improving software development process –> lack of a well-defined process* (8.72), and *good communication between stakeholders –> lack of team communication* (8.64). Applying these practices are essential for curbing their associated causes of TD.

Regarding RQ3 (What are the PARs considered by software practitioners to explain the failure to prevent TD?), this study found 23 PARs that could be used to explain the failure to prevent TD. Among them, *short deadline*, *ineffective management*, *lack of predictability in the software development*, and *requirements change* were the most cited. We realized that PARs can be either an external factor outside the control of the team, related to the customer or the organization for example (an impediment) or a factor used in a decision of the team itself (a decision factor). Failure to prevent TD can be due to either type of PAR. Further, all PARs were grouped into seven categories. The categories *planning and management*, *development issues*, and *methodology* have the largest number of PARs.

We found that *short deadline* (mean of 8.64), *legacy system difficult to heal* (8.36), and *pressure for results* (8.16) were the most important PARs, according to the data from our reduced-scope survey. Software teams can invest effort in resolving these PARs to define a more favorable setting for TD prevention.

### 5.2 Comparison to Related Work

This section compares our work to others with respect to TD prevention practices. It does not include a comparison with respect to PARs as they have not been investigated by others. The section also provides a detailed comparison with our previous study [50] that considers TD prevention practices and PARs for not preventing TD items. Lastly, we do not consider the relationships between TD prevention-related practices and causes of TD in this section, because these have not been previously investigated.

*5.2.1 TD Related-Prevention Practices.* We compared the TD related-prevention practices found in this work with the ones reported in Aragão et al. [9], Apa et al. [12, 13], Yli-Huumo et al. [23],

Table 16. Comparison to Related Work on Categories of TD Prevention-Related Practices

| Our categories | Categories from Li et al. [65] | Overlapping degree |
|---|---|---|
| Development issues | Architecture decision making support | Partial |
| Infrastructure | - | None |
| Internal quality issues | - | None |
| Methodology | Development process improvement | Partial |
| Organizational | - | None |
| People | Human factors | Total |
| Planning and management | Lifecycle cost planning | Partial |

Yli-Huumo et al. [24], Rios et al. [37], Gupta et al. [43], Silva et al. [59], and Codabux et al. [64]. The complete comparison is available in the complementary material [56].

Our work identified 73 new practices, while the related work reported six practices that our work did not recognize. The overlapping is composed of 16 practices: *adoption of good programming practices*, *appropriate tasks allocation*, *code review*, *continuous integration*, *following well-defined project process*, *good communication between stakeholders*, *implementation of a TD management strategy*, *implementation of a TD payment strategy*, *improving documentation*, *improve schedule to include tests and bug fixing*, *improving tests*, *prioritization of TD payment*, *prioritization of test and documentation*, *refactoring*, *training*, and *use the most appropriate version of the technology*. From these practices, *appropriate tasks allocation*, *good communication between stakeholders*, *implementation of a TD management strategy*, *implementation of a TD payment strategy*, *improve schedule to include tests and bug fixing*, *prioritization of TD payment*, and *training* enable TD prevention initiatives, while the others are prevention actions.

We also compared the categories for TD prevention practices identified by Li et al. [65] and those presented in our work. Table 16 presents the comparison results. We can notice that we confirmed the categories *development issues*, *methodology*, *people*, and *planning and management*. The other categories (*infrastructure*, *internal quality issues*, and *organizational*) were only identified in our work.

In summary, our findings on TD prevention-related practices and their categories complement and extend the set of information already reported in the technical literature.

*5.2.2 Comparing the Identified Practices and PARs with Those Reported in Our Previous Study.* Comparing the prevention-related practices reported in this work to those reported in our previous study [50], we have reduced the number of practices by 34%, i.e., from 136 TD prevention practices in Freire et al. [50] to 89 practices in the work presented here. This is due to the coding consolidation we performed while incorporating InsighTD data from the Chilean, Colombian, Costa Rican, and Serbian software industries. In this consolidation, we reviewed all answers collected in Brazil and the United States (used in Freire et al. [50]) and some of them were updated. For example, the practices reported in Freire et al. [50], *well-defined requirements* and *well-defined scope statement* were consolidated on the *well-defined requirements* practice. Table 17 presents the comparison among the 10 most commonly cited practices and their numbers of citations as reported in each study. We can notice that only four practices are common in both lists.

To investigate how similar our list of TD prevention-related practices is to the one reported by Freire et al. [50], we performed the **rank-biased overlap (RBO)** analysis [62], which quantitatively measures how similar the ranked lists are. RBO gives a value ranging from 0 to 1. The closer this value is to 1, the greater the similarity between the lists. As RBO supports top-weighted ranked lists, the first elements of a list have more impact on the similarity index than the last ones. We

Table 17. Comparison among the 10 Most Cited Practices Reported in This and Previous Studies

| Practices from | |
|---|---|
| Our study | Freire et al. [50] |
| Well-defined requirements (57) | Well-defined requirements (24) |
| Adoption of good programming practices (49) | Adoption of good practices (27) |
| Better project management (43) | - |
| Following the project planning (34) | Following the project planning (33) |
| Training (36) | Training (23) |
| Improving software development process (33) | - |
| Improving documentation (26) | - |
| Using good design practices (26) | - |
| Well planned deadlines (26) | - |
| Better project planning (24) | - |
| - | Creating tests (24) |
| - | Code evaluation (21) |
| - | Following well-defined project process (18) |
| - | Risk and impact analysis (17) |
| - | Well-defined architecture (16) |
| - | Refactoring (16) |

The numbers in parentheses indicate the number of citations.

can configure what elements will be compared by setting the $p$-value, which, differently than the p statistic, refers to a level of overlapping and the degree of top-weightedness. In the analysis, we chose $p$-value ranging from 0.5 (only the very initial elements of a rank are considered) to 0.9 (almost all elements are considered). Before performing the analysis, we consolidated the list of practices reported by Freire et al. [50] to assure that the different nomenclature would not influence the comparison.

Figure 10 presents the result of the comparison between the ranked lists of TD prevention-related practices for each study. The RBO analysis (bottom olive line) reveals that the similarity level ranges from 5 to 50% between the two lists. It indicates that the most cited practices in both studies are different, but the similarity increases when more practices are included in the analysis, i.e., the $p$-value increases. Our study complements the previous study, including more evidence of the use of the practices.

Regarding the PARs, we reported 23 PARs here, while our previous study [50] listed seventeen, having an increase of 35%. Some PARs reported in Freire et al. [50] did not appear in this study because of coding consolidation, as described earlier. For example, the PARs, reported in Freire et al. [50], *ineffective management* and *unplanned activities* were consolidated into the *ineffective management* PAR in the current work. Table 18 shows the comparison between all PARs cited in Freire et al. [50] and their correspondence in the present study.

Although all PARs reported in our previous study [50] were confirmed in the present one, we further investigated how similar these lists are by performing an RBO analysis. Figure 10 presents the result of the comparison between the ranked lists of PARs for each study (top blue line). The RBO analysis reveals that the similarity level ranges from 70 to 50% between the two lists. It indicates that the most cited PARs in both studies are similar, and this similarity decreases when more PARs are included in the analysis. Our results complement those reported in our previous study, including more PARs and increasing the evidence of those identified in both studies.

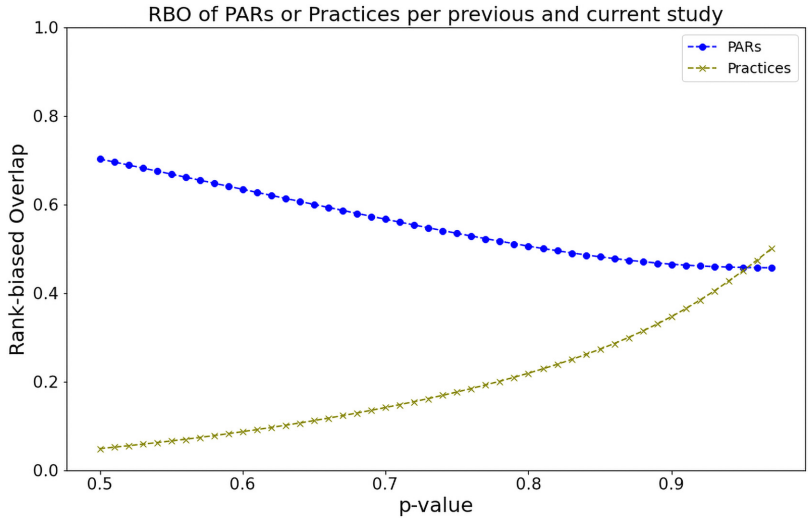



Fig. 10. RBO comparing TD payment-related practices and PARs from our and previous study.

Table 18. Comparison among the PARs Reported in This and Previous Studies

| Practices from | |
|---|---|
| Our study | Freire et al. [50] |
| Short deadline (14) | Short deadline (7) |
| | Need to reduce time to market (2) |
| Lack of concern about maintainability (2) | Lack of concerning about maintainability (2) |
| Lack of qualified professionals (2) | Lack of qualified professionals (2) |
| Lack of technical knowledge (3) | Lack of technical knowledge (2) |
| Ineffective management (7) | Ineffective management (1)Unplanned activities (1) |
| Debt closes to the project end (1) | Debt closes to the project end (1) |
| Documentation issues (lack of or non-updated) (2) | Non-updated documentation (1) |
| | Lack of documentation (1) |
| Lack of good technical solutions (2) | Lack of good technical solutions (1) |
| Requirements change (5) | Requirements change (1) |
| Lack of process maturity (2) | Lack of process maturity (1) |
| Legacy system difficult to heal (2) | Bad design of old systems (1) |
| Restrictions on available infrastructure (1) | Restrictions on available infrastructure (1) |
| Lack of predictability in the software development (5) | Lack of predictability in the software development (1) |
| Continuous change of coding standards (1) | Continuous change of coding standards (1) |

The numbers in parentheses indicate the number of citations.

### 5.3 TD Prevention Conceptual Map

We organized our findings in a map following the concept of evidence briefings [8]. Figure 11 shows our map for supporting development teams in their TD prevention initiatives. The map organizes the TD prevention-related practices and PARs in their respective categories.

Small circles, triangles, rectangles, and diamond next to each practice or PAR name are color and shape-coded to represent the type of practice (prevention action or enabling TD prevention) or PAR

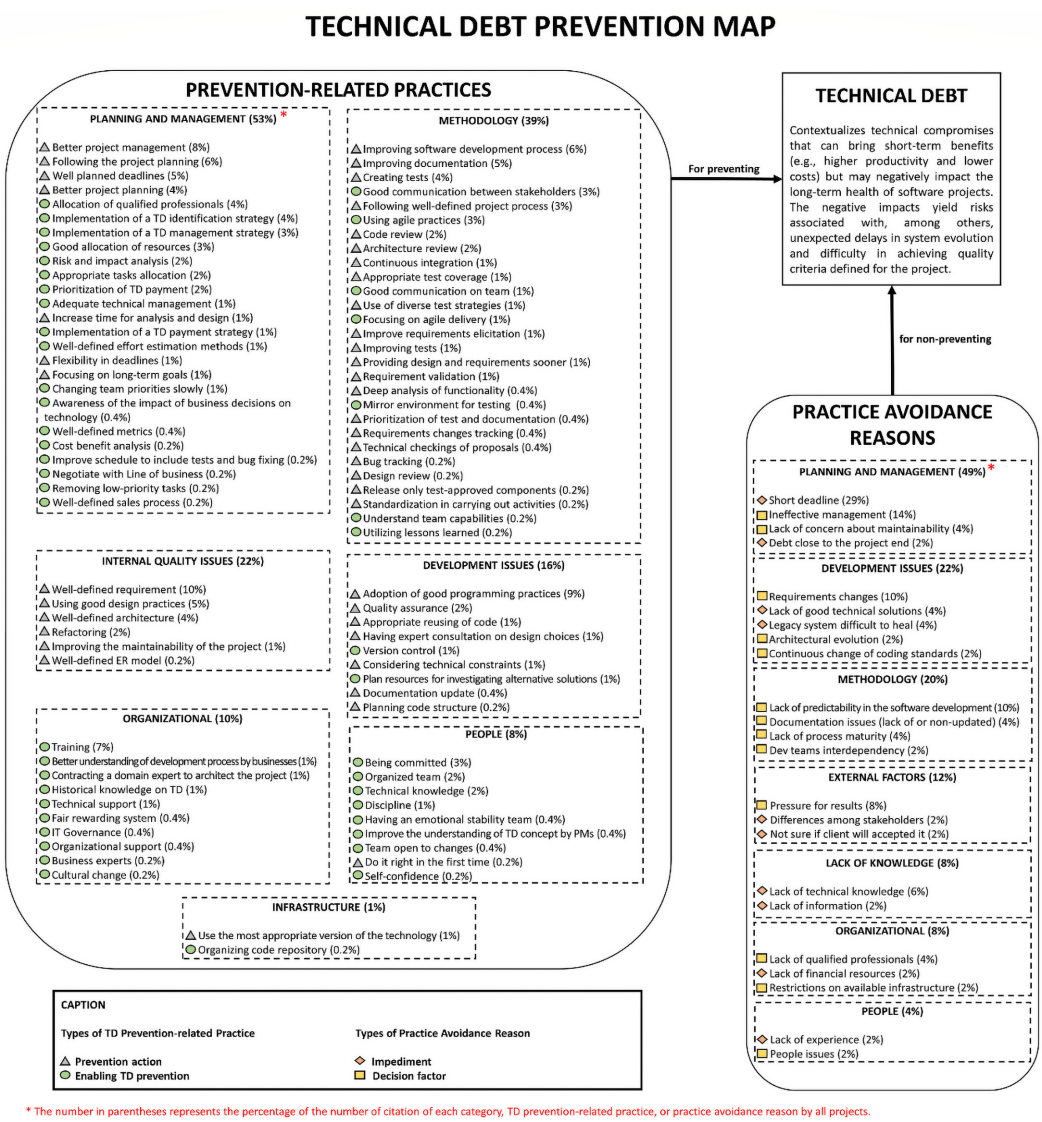



Fig. 11. The TD prevention map.

(decision factor or impediment). We used a specific color and shape for each type. For instance, the practice category *planning and management* has seven gray triangles representing practices (e.g., *better project management*) of the *prevention action* type, and eighteen green circles representing practices (e.g., *allocation of qualified professionals*) of the *enabling TD prevention* type. On the right side, the PAR category *development issues* has two pink diamonds representing PARs (*lack of good technical solutions* and *legacy system difficult to heal*) of the *impediment* type, and three orange rectangles representing PARs (*requirements changes*, *architecture evolution*, and *continuous change of coding standards*) of the *decision factor* type.

By analyzing the practices per categories and types, software practitioners can learn what practices or PARs have been used together for a specific software development concern. For

example, if a development team has experience in TD prevention and using *adoption of good programming practices* for curbing the TD items, they can locate this prevention practice in the *development issues* category, and identify new practices in the same category for (1) preventing the debt (e.g., *documentation update* or *planning code structure*) or (2) increasing its capability for TD prevention, such as, *version control* or *plan resources for investigating alternative solutions*. Similarly, if a development team spent effort to eliminate the *short deadline* PAR from its project, analyzing the PAR *planning and management* category, the team can identify new PARs associated with *short deadline*, such as *ineffective management*, *lack of concern about maintainability*, and *debt close to the project*; and also define mitigation strategies for them.

Each practice, PAR, and category have an associated percentage. To compute it, we summed up the number of occurrences for each of them. After, we divided this value by the total number of projects in which that practice or PAR could have been cited. For example, 43 participants cited the prevention action *better project management*. As we had 546 participants indicating that a TD item could be prevented in their projects, *better project management* was cited in 8% (43/546*100) of them. We performed the same process for computing the percentage of each category.

Looking at the percentages in the map, we see that TD prevention-related practices from the categories *planning and management* and *methodology* are more commonly cited in 53% and 39% of the projects, respectively. In the *planning and management* category the *better project management* prevention action stands out, as it was cited by 8% of the projects. On the right side of the map, we can notice that the PARs from the categories *planning and management* and *development issues* are more cited for explaining the failure to prevent TD in 49% and 22% of the projects, respectively. In the *planning and management* category, the *short deadline* impediment stands out, being cited in 29% of the projects.

Development teams can use the percentages as a criterion for choosing a practice or PAR. For example, if a team has no experience in TD prevention, the team can choose practices with the highest percentages to execute on its projects. Further, the team needs to consider the project context and skills in this choice. This means that, while *training* ranks highest in *organizational* category, the team needs to verify whether its organization can provide staff training.

Lastly, the map can define a favorable environment for preventing the debt, making the practices and PARs visible for all stakeholders involved in a software project. As a communication device, software teams can identify the problems affecting the prevention activities and discuss these problems with their managers and organizations. Then, they can be aware that their decisions are decisive for supporting or not TD prevention initiatives, reducing or increasing the difficulty of preventing TD items. These problems that arise from managers and organizations are depicted in the map from the PAR categories *planning and management* and *organizational*. Besides, the practice categories *planning and management* and *organizational* can hint at possible solutions (practices) for mitigating those PARs. For example, if a team realizes that *lack of qualified professionals* is a PAR existing in its project for the failure to prevent TD, as this PAR is from *organizational* category, the team along with its management can verify what practices from the *organizational* category can be applied to eliminate this PARs, such as the practices *contracting a domain expert to architect the project* and *business experts*.

### 5.4 A Sankey Diagram for Representing the Relationships between TD Prevention Practices and TD Causes

To facilitate the visualization of the results presented in Section 4.4, we organized them into a Sankey diagram [44]. This diagram is composed of nodes and links. A node represents a source, while a link shows a relation between nodes. Further, a link can have a magnitude. The greater this magnitude, the wider the link. In the diagram, the causes and preventive practices are represented as

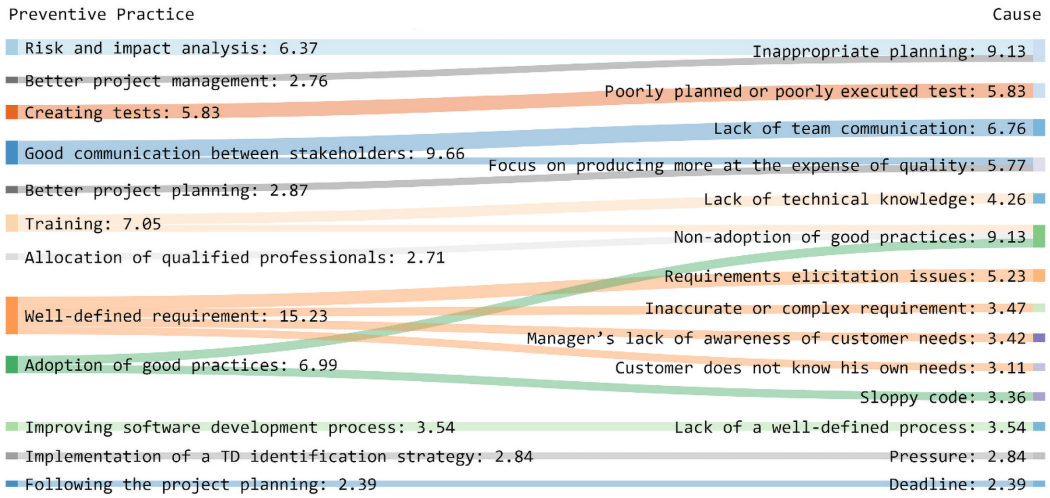



Fig. 12. Sankey diagram for statistically significant relationships between TD causes and preventive practices with at least five occurrences.

nodes and the relationships between them as links. Each link has a magnitude, which corresponds to a RR value associated with the cause and its preventive practice. Thus, the wider the link, the greater the possibility of preventing a TD item in the presence of a cause.

Figure 12 shows the Sankey diagram representing the statistically significant relationships with at least five co-occurrences. We only described these relationships to make the diagram easier to read, considering only the most representative relationships. We can notice, for example, that the preventive practice *training* can be used to increase the possibility of preventing TD items caused by *lack of technical knowledge* and *non-adoption of good practices*. The diagram also reveals that some preventive practices are direct negations of their correspondence causes. For example, *creating tests, adoption of good practices*, and *better project management* preventive practices are indicated to avoid *poorly planned or poorly executed test*, *non-adoption of good practices*, and *inappropriate planning*, respectively.

The Sankey diagram can be useful in two scenarios. First, if a team is starting to manage TD items, the diagram can help them better understand causes and preventive practices based on the experience from others. That is, the links between the preventive practices and the causes are based on the experience of practitioners who deal with debt items. Second, if a team has experience in TD management, the diagram can reveal new causes or preventive practices. Further, in both situations, the teams can use the RR value as a criterion for choosing the most appropriate preventive practice for avoiding TD. For example, if a team is interested in curbing TD items caused by *inappropriate planning*, by consulting the diagram, the team can realize that the practices *risk and impact analysis* and *better project management* are good options. If the team can apply only one of those practices, it could choose the practice *risk and impact analysis* because it has greater RR value, increasing the possibility of preventing TD by almost six times.

## 6 Studies' Validity

As the InsighTD survey is mainly qualitative, this section discusses the survey's trustworthiness according to the criteria (credibility, transferability, dependability, confirmability) described in the

Empirical Standards for Software Engineering [1]. Also, it discusses the threats to reduced-scope survey's validity following the categorization defined by Wohlin et al. [16].

### 6.1 InsighTD Survey

To ensure *credibility*, we used a well stablished research method, where we conducted a survey to collect the data, manual open coding to analyze the qualitative data, and a clear protocol developed by InsighTD experienced researchers. Each InsighTD team was composed of at least one experienced researcher who supervised junior members. Also, we used the convenience sampling method to invite the survey participants, ensuring that they could refuse to participate in the study and guaranteeing the anonymity of the participants. Regarding the data collection, we checked all received answers to guarantee that they fit the three acceptance criteria described in Section 3.3.1, discarding responses that did not comply. Lastly, in the data analysis, two researchers coded the answers, and another researcher resolved the disagreements in all cases, resulting in a high agreement rate (Cohen's Kappa coefficient of 0.822).

To promote *transferability*, i.e., the application of the findings in different contexts [1], we provided the description of the respondents' context, such as company size, country, system size, team size, system age, role, experience rate, and process model. More details are available in Section 4.1.1. Regarding the generalizability of the obtained results, we cannot make strong claims about it because we are not able to estimate the representativeness of our sample given the lack of empirical data characterizing the population. An argument that can be made, however, is that the ecological validity of the work (i.e., the extent to which these findings approximate other real-world scenarios) does hold. Besides, we cannot indicate the generalizability of the results to different types of TD, as the survey did not cover all types defined in the technical literature, such as security [66] and variability [67] debt. Lastly, as the mapping between preventive practices and their corresponding causes was done considering the example given by the participant, it is worth remarking that different examples could lead to different results.

Regarding the *dependability*, we described the application of the research method we used to collect and analyze the data. Also, we provided examples of the data analysis we performed to support the complete understanding of this analysis. Section 3 provides this information, supporting the study replication.

To ensure *confirmability* of the study findings, we provided the research method description in Section 3, enabling the scrutiny of the study findings. Also, we recognize that survey data can be incorrect when participants are not motivated to answer or does not fully understand the survey questions. To mitigate this problem, the participants were familiarized with the study goal and duration, and the participation in the survey was anonymous. Furthermore, the survey questions were submitted to three internal and one external reviews, and a pilot study was performed before the first survey execution. More details on those procedures are provided in Rios et al. [36].

### 6.2 Reduced-Scope Survey

Regarding *construct validity*, a threat arises from the survey's execution and length. As the survey was applied remotely, the participants could misunderstand the questions. Also, the survey is composed of several questions that may lead the participants to drop out in the middle of the survey. To mitigate this threat, we did not include all PARs, prevention-related practices, and their relationships with causes of debt. Instead, we only considered 21 practices (the ten most-cited prevention-related practices and three most common practices per category), 23 PARs, and the top 12 statistically significant relationships between TD causes and practices with at least five occurrences, significantly reducing the number of questions. Then, we performed three validations conducted by researchers from the InsighTD project. They checked the questions for clarity and

completeness and the survey's length. Also, we piloted the survey before its first execution with three participants with different levels of industry experience who were chosen by convenience. These participants looked to give feedback concerning the time to complete the survey (the mean time was about 20 min), the questions (clarity, size, and ease of understanding), and improvement points. None of these participants found the survey challenging to answer. Lastly, we generated the deployment version of the survey.

A *conclusion validity* threat emerges from the fact that the survey did not consider all prevention-related practices and their relations with causes of TD, leading to participants not having a complete view of practices and relations. Some practices and relations not considered could be critical to the participants. To reduce this threat, we chose the most cited practices and the three most recurring practices per category, giving a chance to practices with fewer citations to participate in the study. Also, we included the top 12 statistically significant relationships between practices and causes with at least five occurrences, allowing us to choose only relations with an RR value greater than 2.0, i.e., with a high effect size.

Lastly, concerning the *external validity*, a threat arises from the fact that the participants were chosen for convenience because we invited only practitioners who answered the InsighTD survey, and the number of participants composes a small sample (#25). Even so, the results provide a precious view on the level of importance of, PARs, practices and their relations with causes of TD. The responses are from participants who experienced TD items in their projects, reporting real situations that were considered to describe their causes, how TD items could be preventive and the reasons for the items not being preventive. The quality of this sample can provide a meaningful result, but we recognize that the results are not generalizable, requiring new investigations to gather more evidence on the level of importance of PARs, practices and their relations with causes of TD.

## 7 Final Remarks

This work reports software practitioners' perceptions of TD prevention, discussing the practices cited for avoiding TD items and the PARs cited for explaining the failure to prevent TD. Also, we investigate the co-occurrence between preventive practices and causes of TD. Practitioners can use the list of preventive practices to support their strategies for avoiding potential TD items in their projects. The list of PARs can support software practitioners in increasing their ability to curb TD items. Furthermore, the relationship between causes and preventive practices contributes to defining the most appropriate practice that can be applied given the identification of specific causes. Lastly, this work prioritizes practices, relationships, and PARs regarding their importance for TD prevention by collecting software practitioners' opinions.

To make our results more beneficial to use in practice, we organize the set of practices and PARs in a conceptual map. This can support software practitioners in identifying practices and PARs grouped by categories and types. By analyzing per category, software practitioners can identify practices or PARs that can be used or found together concerning a software development issue. On the other side, by analyzing per type, software practitioners can identify i) practices that could be used to prevent the debt or increase the teams' ability for TD prevention, and ii) PARs related to the decisions made by a software team or from external factors that are out of team's control. Furthermore, we organize the relationships between practices and causes in a Sankey diagram. This can guide software practitioners in choosing practices for avoiding a specific cause of debt.

Although our findings can support TD prevention initiatives, software professionals should be aware that their projects and organizational contexts are decisive as they influence the decision of which practice to implement and how to mitigate PARs.

For researchers, our findings can motivate new investigations aligned with the TD prevention state of the practice. The list of practices, PARs, and relationships between practices and causes can drive new pieces of research in a problem-driven way. Moreover, the map and the Sankey diagram can support the definition of new strategies for TD prevention.

The reviewers of this article raised the issue of how we would classify our qualitative analysis method. Our work involved thematic analysis [21], as there was a portion of inductive analysis and concept development in our analyses. Cases in point were the development of the concepts of Types and Categories of TD prevention practices and PARs. However, our work is largely based on content analysis [26, 45] as our analytical process was mostly deductive, including the combination of qualitative and quantitative analysis based on preestablished TD concepts and a TD management process framework drawn from previous TD studies [33, 36, 47].

In future work, we intend to empirically assess the map and the Sankey diagram regarding their support of TD prevention activities. Besides, we seek to perform analyses to investigate whether the co-occurrence between causes and prevention practices is influenced by other variables like the type of debt, adopted process model, participant experience and role, and organization/project size.

## References


[1] ACM Special Interest Group on Software Engineering. ACM SIGSOFT Empirical Standards for Software Engineering. Retrieved from https://github.com/acmsigsoft/EmpiricalStandards

[2] Anselm Strauss and Juliet Corbin. 1998. *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory*. Sage Publications, Thousand Oaks, CA, XIII.

[3] Antonio Martini, Jan Bosch, and Michel Chaudron. 2014. Architecture technical debt: Understanding causes and a qualitative model. In *Proceedings of the 40th Conference on Software Engineering and Advanced Applications*. 85–92.

[4] Alexia Pacheco, Gabriela Marín-Raventós, and Gustavo López. 2019. Technical debt in Costa Rica: An InsighTD survey replication. In *Product-Focused Software Process Improvement*. X. Franch, T. Männistö, and S. Martínez-Fernández (Eds.). PROFES 2019. Lecture Notes in Computer Science, Vol. 11915, Springer, Cham, 1–10. DOI: https://doi.org/10.1007/978-3-030-35333-9_17

[5] Boris Pérez, Juan Pablo Brito, Hernán Astudillo, Darío Correal, Nicolli Rios, Rodrigo O. Spínola, Manoel Mendonça, and Carolyn Seaman. 2019. Familiarity, causes and reactions of software practitioners to the presence of technical debt: A replicated study in the chilean software industry. In *Proceedings of 38th International Conference of the Chilean Computer Science Society (SCCC)*. 1–7. DOI: https://doi.org/10.1109/SCCC49216.2019.8966424

[6] Bóris Pérez, Camilo Castellanos, Darío Correal, Nicolli Rios, Sávio Freire, Rodrigo Spínola, and Carolyn Seaman. 2020. What are the practices used by software practitioners on technical debt payment? Results from an international family of surveys. In *Proceedings of the 3rd International Conference on Technical Debt (TechDebt '20)*. ACM, New York, NY, 103–112. DOI: https://doi.org/10.1145/3387906.3388632

[7] Bóris Pérez, Camilo Castellanos, Darío Correal, Nicolli Rios, Sávio Freire, Rodrigo Spínola, Carolyn Seaman, Clemente Izurieta. 2021. Technical debt payment and prevention through the lenses of software architects. *Information and Software Technology*, 140 (2021), 106692. DOI: https://doi.org/10.1016/j.infsof.2021.106692

[8] Bruno Cartaxo, Gustavo Pinto, Elton Vieira, and Sérgio Soares. 2016. Evidence briefings: towards a medium to transfer knowledge from systematic reviews to practitioners. In *Proceedings of the 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '16)*. ACM, New York, NY, Article 57, 1–10. DOI: https://doi.org/10.1145/2961111.2962603

[9] Bruno Aragão, Rossana Andrade, Ismayle Santos, Rute Castro, Valeria Lelli, and Ticianne Darin. 2021. TestDCat 3.0: catalog of test debt subtypes and management activities. *Software Quality Journal*. 30, 181–225. DOI: https://doi.org/10.1007/s11219-020-09533-y

[10] Carolyn Seaman. 1999. Qualitative methods in empirical studies of software engineering. *IEEE Transactions on Software Engineering* 25, 4 (1999), 557–572.

[11] Carolyn Seaman, Yuepu Guo, Nico Zazworka, Forrest Shull, Clemente Izurieta, Yuanfang Cai, and Antonio Vetrò. 2012. Using technical debt data in decision making: potential decision approaches. In *Proceedings of the 3rd International Workshop on Managing Technical Debt (MTD)*. 45–48. DOI: https://doi.org/10.1109/MTD.2012.6225999

[12] Cecilia Apa, Martin Solari, Diego Vallespir, and Guilherme Travassos. 2020. A taste of the software industry perception of technical debt and its management in uruguay: A survey in software industry. In *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 1–9.

[13] Cecilia Apa, Helvio Jeronimo, Luciana Nascimento, Diego Vallespir, and Guilherme Travassos. 2020. The perception and management of technical debt in software startups. In *Fundamentals of Software Startups*. A. Nguyen-Duc, J. Münch, R. Prikladnicki, X. Wang, and P. Abrahamsson (Eds.), Springer, Cham, 61–78.

[14] Chittaranjan Andrade. 2015. Understanding relative risk, odds ratio, and related terms: As simple as it can get. *The Journal of Clinical Psychiatry* 76, 7 (2015), 21865.

[15] Chittaranjan Andrade. 2018. Internal, external, and ecological validity in research design, conduct, and evaluation. *Indian Journal of Psychological Medicine* 40, 5 (2018), 498–499. DOI: https://doi.org/10.4103/IJPSYM.IJPSYM_334_18

[16] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering: An Introduction*. Springer.

[17] Clara Berenguer, Adriano Borges, Sávio Freire, Nicolli Rios, Nebojša Taušan, Robert Ramač, Boris Pérez, Camilo Castellanos, Darío Correal, Alexia Pacheco, Gustavo López, Davide Falessi, Carolyn Seaman, Vladimir Mandić, Clemente Izurieta, and Rodrigo Spínola. 2021. technical debt is not only about code and we need to be aware about it. In *Proceedings of the XX Brazilian Symposium on Software Quality (SBQS '21)*. ACM, New York, NY, Article 14, 1–12. DOI: https://doi.org/10.1145/3493244.3493285

[18] Clara Berenguer, Adriano Borges, Sávio Freire, Nicolli Rios, Robert Ramač, Nebojša Taušan, Boris Pérez, Camilo Castellanos, Darío Correal, Alexia Pacheco, Gustavo López, Manoel Mendonça, Davide Falessi, Carolyn Seaman, Vladimir Mandić, Clemente Izurieta, and Rodrigo Spínola. 2023. Investigating the Relationship between Technical Debt Management and Software Development Issues. In *Journal of Software Engineering Research and Development* 11, 1 (2023), 3–1.

[19] Clemente Izurieta, Antonio Vetrò, Nico Zazworka, Yuanfang Cai, Carolyn Seaman, and Forrest Shull. 2012. Organizing the technical debt landscape. In *Proceedings of the 3rd International Workshop on Managing Technical Debt (MTD '12)*. 23–26.

[20] Clemente Izurieta, Ipek Ozkaya, Carolyn Seaman, Philippe B. Kruchten, Robert L. Nord, Will Snipes, and Paris Avgeriou. 2016. Perspectives on managing technical debt: A transition point and roadmap from Dagstuhl. In *Proceedings of the 1st International Workshop on Technical Debt Analytics (TDA). Association with the 23rd Asia-Pacific Software Engineering Conference (APSEC)*, University of Waikato, Hamilton, New Zealand, 1–12.

[21] Daniela Soares Cruzes and Tore Dyba, recommended steps for thematic synthesis in software engineering. In *Proceedings of the 2011 International Symposium on Empirical Software Engineering and Measurement*. 275–284. DOI: https://doi.org/10.1109/ESEM.2011.36

[22] Davide Falessi and Rick Kazman. 2021. Worst smells and their worst reasons. In *Proceedings of the IEEE/ACM International Conference on Technical Debt (TechDebt)*. 45–54. DOI: https://doi.org/10.1109/TechDebt52882.2021.00014

[23] Jesse Yli-Huumo, Andrey Maglyas, and Kari Smolander. 2014. The sources and approaches to management of technical debt: A case study of two product lines in a middle-size finnish software company. In *Product-Focused Software Process Improvement*. A. Jedlitschka, P. Kuvaja, M. Kuhrmann, T. Männistö, J. Münch, and M. Raatikainen (Eds.), PROFES 2014, Lecture Notes in Computer Science, Vol. 8892. Springer, Cham, 93–107.

[24] Jesse Yli-Huumo, Andrey Maglyas, and Kari Smolander. 2016. How do software development teams manage technical debt? - An empirical study. *Journal of Systems and Software* 120, C (2016), 195–218.

[25] Jiakun Liu, Qiao Huang, Xin Xia, Emad Shihab, David Lo, and Shanping Li. 2020. Is using deep learning frameworks free? characterizing technical debt in deep learning frameworks. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Society*, IEEE. 1–10.

[26] Klaus Krippendorff. 1980. *Content Analysis: An Introduction to Its Methodology*. Sage, Beverly Hills, CA.

[27] Larissa Barbosa, Sávio Freire, Nicolli Rios, Robert Ramač, Nebojša Taušan, Boris Pérez, Camilo Castellanos, Darío Correal, Alexia Pacheco, Gustavo López, Vladimir Mandić, Rita S. PMaciel, Manoel Mendonça, Davide Falessi, Clemente Izurieta, Carolyn Seaman, and Rodrigo Spínola. 2022. Organizing the TD management landscape for requirements and requirements documentation debt. In *Proceedings of the 25th Workshop on Requirements Engineering (WER '22)*, 35–44.

[28] Lucinéia Souza, Sávio Freire, Verusca Rocha, Nicolli Rios, Rodrigo O. Spínola and Manoel Mendonça. 2020. Using surveys to build-up empirical evidence on test-related technical debt. In *Proceedings of the 34th Brazilian Symposium on Software Engineering (SBES '20)*. ACM, New York, NY, 750–759. DOI: https://doi.org/10.1145/3422392.3422430

[29] Mary L. McHugh. 2012. Interrater Reliability: The Kappa Statistic. *Biochemia Medica* 22, 3 (2012), 276–282.

[30] Neil Ernst, Stephany Bellomo, Ipek Ozkaya, Robert Nord, and Ian Gorton. 2015. Measure it? Manage it? Ignore it? software practitioners and technical debt. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 50–60.

[31] Nicolli Rios, Rodrigo Spínola, Manoel Mendonça, and Carolyn Seaman. 2018. A study of factors that Lead development teams to incur technical debt in software projects. In *Proceedings of the 44th Euromicro Conference on Software Engineering and Advanced Applications*. 429–436.

[32] Nicolli Rios, Rodrigo Spínola, Manoel Mendonça, and Carolyn Seaman. 2018. The most common causes and effects of technical debt: first results from a global family of industrial surveys. In *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '18)*. ACM, New York, NY, Article 39, 1–10. DOI: https://doi.org/10.1145/3239235.3268917

[33] Nicolli Rios, Manoel Mendonça Neto, and Rodrigo O. Spínola. 2018. A tertiary study on technical debt: Types, management strategies, research trends, and base information for practitioners. *Information and Software Technology* 102, June (2018), 117–145 DOI: https://doi.org/10.1016/j.infsof.2018.05.010

[34] Nicolli Rios, Manoel Mendonça, Carolyn Seaman, and Rodrigo O. Spínola. 2019. Causes and effects of the presence of technical debt in agile software projects. In *Proceedings of the Americas Conference on Information Systems (AMCIS)*, 1–10.

[35] Nicolli Rios, Rodrigo O. Spínola, Manoel Mendonça, and Carolyn Seaman. 2019. Supporting analysis of technical debt causes and effects with cross-company probabilistic cause-effect diagrams. In *Proceedings of the 2nd International Conference on Technical Debt (TechDebt)*. IEEE Press, 3–12. DOI: https://doi.org/10.1109/TechDebt.2019.00009

[36] Nicolli Rios, Rodrigo O. Spínola, Manoel Mendonça, and Carolyn Seaman. 2020. The practitioners' point of view on the concept of technical debt and its causes and consequences: A design for a global family of industrial surveys and its first results from Brazil. *Empirical Software Engineering* 25 (2020), 3216–3287. DOI: https://doi.org/10.1007/s10664-020-09832-9

[37] Nicolli Rios, Leonardo Mendes, Cristina Cerdeiral, Ana P.F. Magalhães, Boris Pérez, Darío Correal, Hernán Astudillo, Carolyn Seaman, Clemente Izurieta, Gleison Santos, and Rodrigo O. Spínola. 2020. Hearing the voice of software practitioners on causes, effects, and practices to deal with documentation debt. In *Requirements Engineering: Foundation for Software Quality*. N. Madhavji, L. Pasquale, A. Ferrari, and S. Gnesi (Eds.), REFSQ 2020, Lecture Notes in Computer Science, Vol. 12045. Springer, Cham, 55–70. DOI: https://doi.org/10.1007/978-3-030-44429-7_4

[38] Nicolli Rios, Sávio Freire, Boris Pérez, Camilo Castellanos, Dario Correal, Manoel Mendonça, Davide Falessi, Clemente Izurieta, Carolyn Seaman, and Rodrigo Oliveira Spínola. 2021. On the relationship between technical debt management and process models. *IEEE Software* 38, 5 (2021), 56–64.

[39] Noopur Davis. 2013. Driving quality improvement and reducing technical debt with the definition of done. In *Proceedings of the Agile Conference*. 164–168. DOI: https://doi.org/10.1109/AGILE.2013.21

[40] Paris Avgeriou, Philippe Kruchten, Ipek Ozkaya, and Carolyn Seaman. 2016. Managing technical debt in software engineering (Dagstuhl Seminar 16162). *Dagstuhl Reports* 6, 4 (2016), 110–138.

[41] Paris Avgeriou, Philippe Kruchten, Robert L. Nord, Ipek Ozkaya and Carolyn Seaman. 2016. Reducing friction in software development. *IEEE Software* 33, 1 (2016), 66–73. DOI: https://doi.org/10.1109/MS.2016.13

[42] Project Management Institute. 2017. *A Guide to the Project Management Body of Knowledge: PMBOK Guide* (6th. ed.). Newtown Square, PA.

[43] Rajeev Gupta, Prabhulinga Manikreddy, Sandesh Naik, and K. Arya. 2016. Pragmatic approach for managing technical debt in legacy software project. In *Proceedings of the 9th India Software Engineering Conference*. 170–176.

[44] Richard C. Lupton and Julian M. Allwood. 2017. Hybrid Sankey diagrams: Visual analysis of multidimensional data for understanding resource use. *Resources, Conservation & Recycling* 124 (2017), 141–151. DOI: https://doi.org/10.1016/j.resconrec.2017.05.002

[45] Robert Philip Weber. 1990. *Basic Content Analysis*, Vol. 49. Sage.

[46] Robert Ramač, Vladimir Mandić, Nebojša Taušan, Nicolli Rios, Manoel Mendonça, Carolyn Seaman, and Rodrigo O. Spínola. 2020. Common causes and effects of technical debt in Serbian IT: InsighTD Survey replication. In *Proceedings of the Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. 354–361. DOI: https://doi.org/10.1109/SEAA51224.2020.00065

[47] Robert Ramač, Vladimir Mandić, Nebojša Taušan, Nicolli Rios, Sávio Freire, Boris Pérez, Camilo Castellanos, Darío Correal, Alexia Pacheco, Gustavo López, Clemente Izurieta, Carolyn Seaman, and Rodrigo Spínola. 2022. Prevalence, common causes and effects of technical debt: Results from a family of surveys with the IT industry. *Journal of Systems and Software* 184 (2022), 111114. DOI: https://doi.org/10.1016/j.jss.2021.111114

[48] Robert Ramač, Nebojša Taušan, Sávio Freire, Nicolli Rios, Manoel G. de Mendonça Neto, Rodrigo Oliveira Spínola, and Vladimir Mandić. 2022. Technical debt payment practices and rationales behind payment avoidance in the Serbian IT industry. In *Proceedings on 18th International Conference on Industrial Systems (IS '20)*. B. Lalic, D. Gracanin, N. Tasic, and N. Simeunović (Eds.), Lecture Notes on Multidisciplinary Industrial Engineering. Springer, Cham, 60–68. DOI: https://doi.org/10.1007/978-3-030-97947-8_14

[49] Rodrigo Spínola, Nico Zazworka, Antonio Vetrò, Forrest Shull, and Carolyn Seaman. 2019. Understanding automated and human based technical debt identification approaches-a two-phase study. *Journal of the Brazilian Computer Society* 25, 5 (2019), 1–21.

[50] Sávio Freire, Nicolli Rios, Manoel Mendonça, Davide Falessi, Carolyn Seaman, Clemente Izurieta, and Rodrigo Spínola. 2020. Actions and impediments for technical debt prevention: results from a global family of industrial surveys. In *Proceedings of the 35th Annual ACM Symposium on Applied Computing (SAC '20)*. 1548–1555.

[51] Sávio Freire, Nicolli Rios, Boris Gutierrez, Darío Torres, Manoel Mendonça, Clemente Izurieta, Carolyn Seaman, and Rodrigo O. Spínola. 2020. Surveying software practitioners on technical debt payment practices and reasons for not paying off debt items. In *Proceedings of the Evaluation and Assessment in Software Engineering (EASE '20)*. ACM, New York, NY, 210–219. DOI: https://doi.org/10.1145/3383219.3383241

[52] Sávio Freire, Nicolli Rios, Boris Pérez, Camilo Castellanos, Darío Correal, Robert Ramač, Vladimir Mandić, Nebojša Taušan, Gustavo López, Alexia Pacheco, Davide Falessi, Manoel Mendonça, Clemente Izurieta, Carolyn Seaman, and Rodrigo Spínola. 2021. How experience impacts practitioners' perception of causes and effects of technical debt. In *Proceedings of the IEEE/ACM 13th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*. IEEE. 21–30. DOI: https://doi.org/10.1109/CHASE52884.2021.00011

[53] Sávio Freire, Nicolli Rios, Boris Pérez, Camilo Castellanos, Darío Correal, Robert Ramač, Nebojša Taušan, Vladimir Mandić, Alexia P. Hernandez, Gustavo López, Manoel Mendonça, Clemente Izurieta, Davide Falessi, Carolyn B. Seaman, Rodrigo O. Spínola. 2021. Pitfalls and solutions for technical debt management in agile software projects. *IEEE Software* 38, 6 (2021), 42–49. DOI: https://doi.org/10.1109/MS.2021.3101990

[54] Sávio Freire, Nicolli Rios, Boris Pérez, Darío Correal, Manoel Mendonça, Clemente Izurieta, Carolyn Seaman and Rodrigo Spínola. 2021. How do technical debt payment practices relate to the effects of the presence of debt items in software projects? In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '21)*. IEEE, 605–609. DOI: https://doi.org/10.1109/SANER50967.2021.00074

[55] Sávio Freire, Nicolli Rios, Boris Pérez, Camilo Castellanos, Darío Correal, Robert Ramač, Vladimir Mandić, Nebojša Taušan, Gustavo López, Alexia Pacheco, Manoel Mendonça, Davide Falessi, Clemente Izurieta, Carolyn Seaman, and Rodrigo Spínola. 2023. Software practitioners' point of view on technical debt payment. *Journal of Systems and Software* 196 (2023), 111554, 137–177. DOI: https://doi.org/10.1016/j.jss.2022.111554

[56] Sávio Freire, Alexia Pacheco, Nicolli Rios, Boris Pérez, Camilo Castellanos, Darío Correal, Robert Ramač, Vladimir Mandić, Nebojša Taušan, Gustavo López, Manoel Mendonça, Davide Falessi, Clemente Izurieta, Carolyn Seaman, and Rodrigo Spínola. 2024. A comprehensive view on TD prevention practices and reasons for not preventing it: complementary material [Data set]. *Zenodo*. DOI: https://doi.org/10.5281/zenodo.10729979

[57] Steve McConnell. 2007. Technical debt, 10x Software Development Blog. *Construx Conversations*. Retrieved from http://blogs.construx.com/blogs/stevemcc/archive/2007/11/01/technical-debt-2.aspx

[58] Verusca Rocha, Sávio Freire, Nicolli Rios, Cleydiane Lima, Leilane Ribeiro, Boris Pérez, Arilo Dias Neto, Hermano Moura, Dario Correal, Manoel Mendonça, and Rodrigo Spínola. 2021. A conceptual framework to support the management of technical debt in software testing. In *Proceedings of the Americas Conference on Information Systems (AMCIS)*, 100–112.

[59] Victor Machadoda Silva, Helvio Jeronimo Junior, and Guilherme Horta Travassos. 2019. A taste of the software industry perception of technical debt and its management in Brazil. *Journal of Software Engineering Research and Development* 7 (2019), 1:1-1:16. DOI: https://doi.org/10.5753/jserd.2019.19

[60] Vladimir Mandić, Nebojša Taušan, and Robert Ramač. 2020. The prevalence of the technical debt concept in Serbian IT industry: results of a national-wide survey. In *Proceedings of the 3rd International Conference on Technical Debt (TechDebt)*. ACM, New York, NY, 77–86. DOI: https://doi.org/10.1145/3387906.3388622

[61] Vladimir Mandić, Nebojša Taušan, Robert Ramač, Sávio Freire, Nicolli Rios, Boris Pérez, Camilo Castellanos, Darío Correal, Alexia Pacheco, Gustavo López, Clemente Izurieta, Davide Falessi, Carolyn Seaman, and Rodrigo Spínola. 2021. Technical and nontechnical prioritization schema for technical debt: Voice of TD-experienced practitioners. *IEEE Software* 38, 6 (2021). 50–58. DOI: https://doi.org/10.1109/MS.2021.3103121

[62] William Webber, Alistair Moffat, and Justin Zobel. 2010. A similarity measure for indefinite rankings. *ACM Transactions on Information Systems* 28, 4 (2010), 1–38.

[63] Zadia Codabux and Byron Williams. 2013. Managing technical debt: an industrial case study. In *Proceedings of the 4th International Workshop on Managing Technical Debt (MTD)*. 8–15.

[64] Zadia Codabux, Byron Williams, and Nan Niu. 2014. A quality assurance approach to technical debt. In *Proceedings of the International Conference on Software Engineering Research and Practice (SERP)*, 60–68.

[65] Zengyang Li, Paris Avgeriou, and Peng Liang. 2015. A systematic mapping study on technical debt and its management. *Journal of Systems and Software* 101 (2015), 193–220.

[66] Jabier Martinez, Nuria Quintano, Alejandra Ruiz, Izaskun Santamaria, Iker M. de Soria, and José Arias. 2021. Security debt: characteristics, product life-cycle integration and items. In *Proceedings of the IEEE/ACM International Conference on Technical Debt (TechDebt)*. 1–5. DOI: https://doi.org/10.1109/TechDebt52882.2021.00009

[67] Daniele Wolfart, Jabier Martinez, Wesley K. G. Assunção, Thelma E. Colanzi, and Alexander Egyed. 2024. Variability debt in opportunistic reuse: A multi-project field study. *Journal of Systems and Software* 210 (2024), 111969. DOI: https://doi.org/10.1016/j.jss.2024.111969

[68] Mario Farias, Jose Amancio Santos, Marcos Kalinowski, Manoel Mendonça, Rodrigo Spínola. 2017. Investigating the identification of technical debt through code comment analysis. In *Enterprise Information Systems*. S. Hammoudi, L. Maciaszek, M. Missikoff, O. Camp, and J. Cordeiro (Eds.), ICEIS 2016, Lecture Notes in Business Information Processing, Vol. 291. Springer, Cham, 110–126. DOI: https://doi.org/10.1007/978-3-319-62386-3_14

Received 3 July 2023; revised 2 March 2024; accepted 12 May 2024